
Optimization Summary

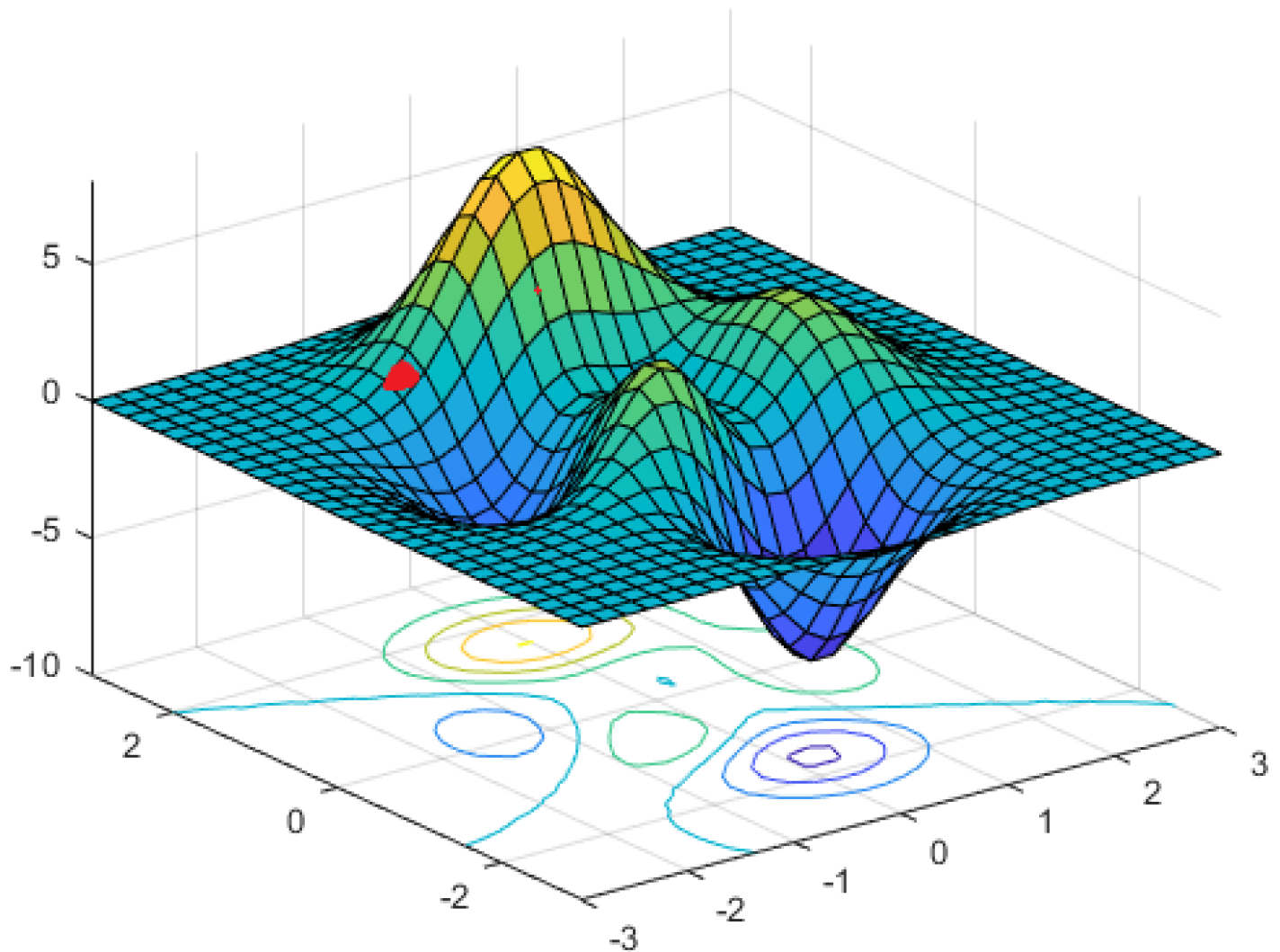
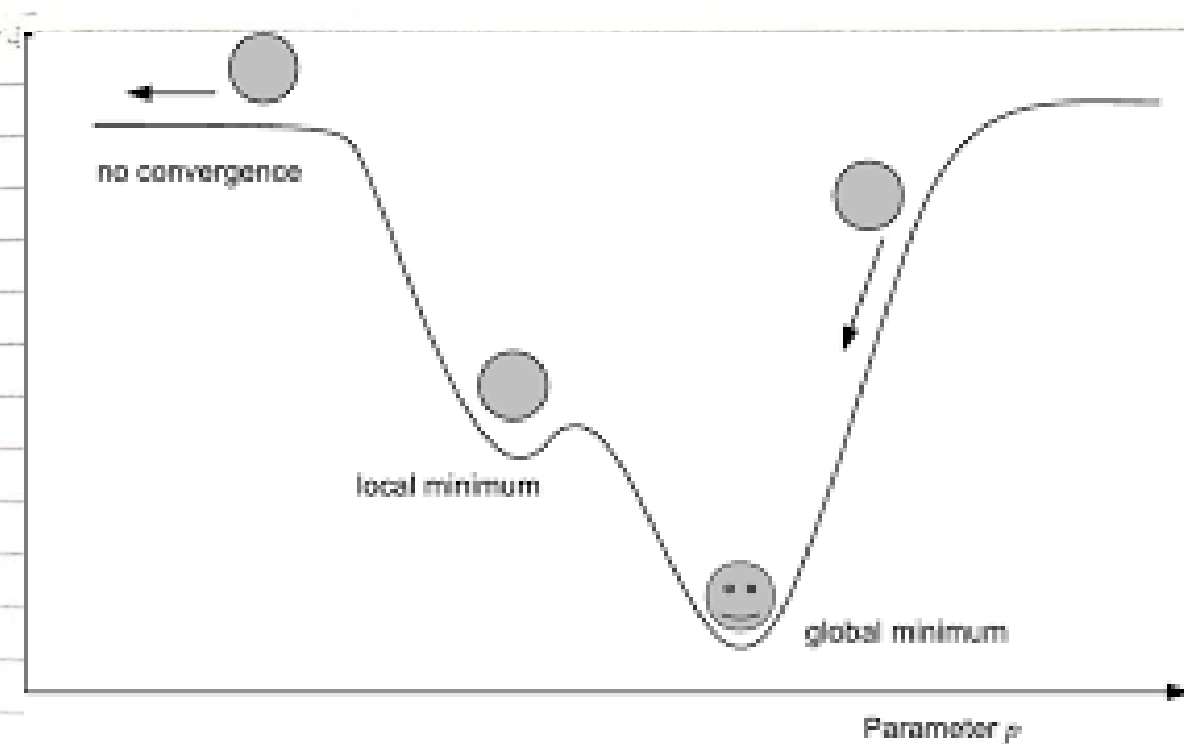


Table Of Content:

- **Introduction to Optimization**
- **Steps in Numerical Optimization**
- **Effect of Learning Rate**
- **Hyperparameters**
- **Gradient Vector in Single-Variable Linear Regression**
- **Gradient Vector in Multiple-Variable Linear Regression**
- **Feature Scaling**
 - **Min – Max Normalization**
 - **Mean Normalization**
 - **Robust Scaling**
- **Limitations of Gradient Descent**
- **Training Cycle**
- **Gradient Descent Types:**
 - **Batch Gradient Descent (BGD)**
 - **Stochastic Gradient Descent (SGD)**
 - **Mini-Batch Gradient Descent (MBGD)**
- **Random Shuffle**
- **Momentum-Based Gradient Descent**
- **Nesterov Accelerated Gradient (NAG)**
- **Adaptive Gradient (AdaGrad)**
- **Root Mean Square Propagation (RMSProp)**
- **Adaptive Moment Estimation (Adam)**
- **Bias Correction in Adam**
- **Connection of Gradient Descent to Taylor Series**
- **Second-Order Optimization**
- **Quasi-Newton Methods**
- **BFGS Algorithm**

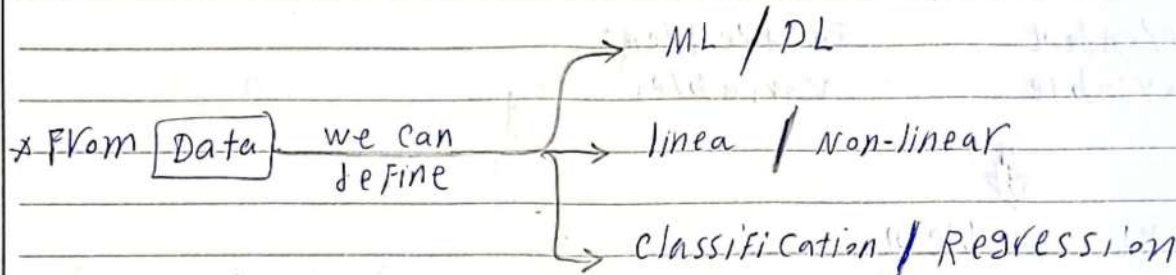
Numerical optimization

- is a branch of mathematics focused on finding (the best solution) to a problem among many possible solutions.
- It's about minimizing/maximizing an objective function under given conditions (constraints).



* Introduction:

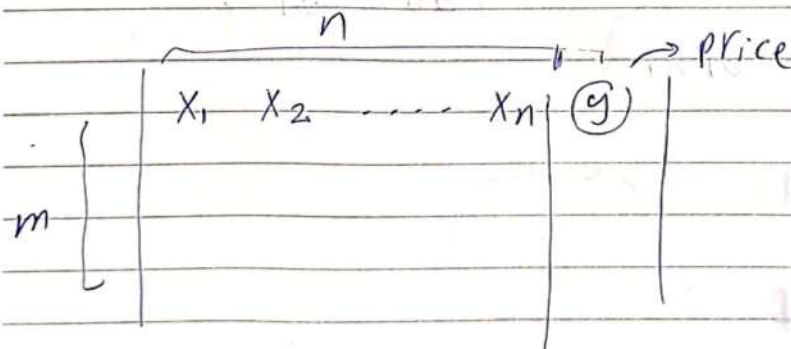
→ Every problem starts with Data.



* data بتيجي عبارة عن (samples)
 ← كل مال quality كانت اتمن كل مال model كذا

ex:

- to build ML Model:
 prediction → house pricing



* To find Relation between Features & label:

$$y = f(x_1, x_2, \dots, x_n)$$

Dependent
variable

Independent
variables



ML problem:

$$\hat{y} = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

Linear



Using ML $\Rightarrow$ we find the parameters

θ_0 θ_1

$$(y = \theta_0 + \theta_1 x_1)$$



* IF the relation is Non-linear:

⇒ when using (logistic Regression) For Example.

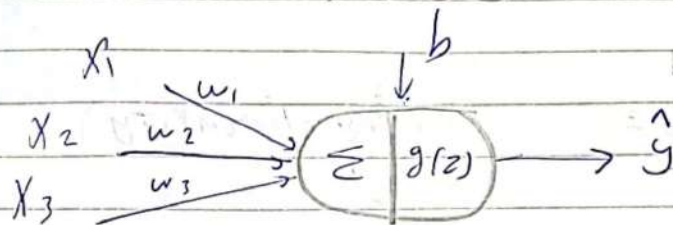
$$Z = \theta_1 X_1 + \theta_2 X_2 + \dots + \theta_n X_n$$

$$\hat{y} = \frac{1}{1 + e^{-Z}} \rightarrow (\text{Non-linearity})$$



* when :
 → out Data is very large
 → more complex
 → has Non-linear Relation
 → we need high performance.

then, we use NN

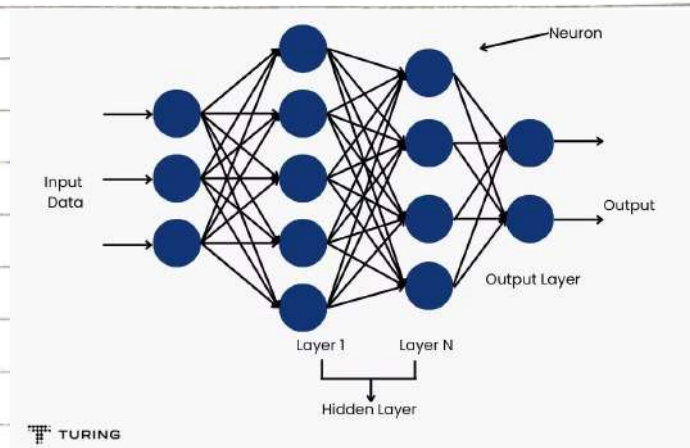


$$Z = b + w_1 x_1 + \dots + w_n x_n$$

$$\hat{y} = g(Z)$$

we need to find the optimum values $\left\langle \begin{matrix} w \\ b \end{matrix} \right\rangle$

بحسب دخلي $\hat{y}$ أفضل ما يمكن



* How to find parameters :-

↳ In Training phase

x_1	x_2	...	x_n	y	$\hat{y}$

$h(x)$
predict
"hypothesis
Function"

Example :

	x_1	x_2	y
	5	1	100
A	6	2	150

To find θ_1, θ_2 :

$$5\theta_1 + 1\theta_2 = 100$$

$$6\theta_1 + 2\theta_2 = 150$$

$$\begin{bmatrix} 5 & 1 \\ 6 & 2 \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} = \begin{bmatrix} 100 \\ 150 \end{bmatrix}$$

$$X \cdot \theta = y$$

$$\theta = X^{-1} y \Rightarrow \text{Definite}$$

((Analytical solution))

doesn't make generalization of the model



x لو جربت أضف inputs جديدة بعد ما وصلت

Analytical parameters بار parameters بار

Solution

	x_1	x_2	y
→	5	2	100
	0	1	150
	7	3	200

new

predict

القيمة بعدة
تغير عن الـ actual
(error)

problems

- The model not generalized
- over fitting
- High error ↑↑

* To Find the solution By Algebraic solution

⇒ We should use ⇒ system of normal equations

$$m > n$$

"over determined"

Least square

$$m < n$$

"under determined"

Least Norm



① least square : ($m > n$)

→ No. of equations $>$ No. of Variables

بما أن عدد المعادلات أكبر من عدد المتغيرات، فإننا نحاول إيجاد θ الذي يقرب من حل المعادلات بأقل (error) ممكنة.

$$\theta_{LS} = \min_{\theta} \|X\theta - y\|_2^2$$

$$\theta = (X^T X)^{-1} X^T Y$$

② least Norm : ($m < n$)

→ No. of equations (rows) $<$ No. of Variables (parameters)
له عدد من الحلول.

$$\theta_{LN} = \arg \min_{\theta} \|\theta\|_2$$

$$\theta_{LN} = X^T (X X^T)^{-1} Y$$



* the limitations of Analytical solution:

① Not suitable for Non-linear relations

② Computational Complexity:

↳ For compute $(X^T X)^{-1}$

↳ $\Rightarrow$ complexity = $O(n^3)$

n : No. of features

③ Numerical stability

$\Rightarrow (X^T X)$ may be singular

④ Memory usage:

$X^T X \Rightarrow n \times n \Rightarrow$ Very large memory usage

so, we use Numerical solution

لأنه يجب أن تكون error أقل ما يمكن



* Steps to use Numerical optimization:

① Initialization

* assume initial parameter: $(\theta_1, \theta_2, \dots, \theta_n)$

② $h_{\theta}(x) = \hat{y} = \theta x$ (model)

③ Evaluate the prediction

Cost function
loss
error

$$MAE = \frac{1}{m} \sum_{i=1}^m |\hat{y}_i - y_i|$$

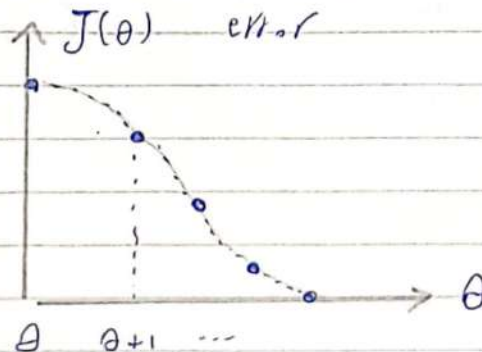
$$MSE = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

④ Define

→ Direction of change

→ Amount of change

⑤ Update θ : $\theta_{t+1} = \theta_t + \text{Amount}$



* في حالة، استفسار (Numerical solution)

بلجاً لا iterations

(More iterations) = more cost

* We want to reach min error in less no. of iterations.

* Evaluation :

له تقييم أداء ال (Model) عن طريق معرفة مقدار ال (error) $|y - \hat{y}|$

We use : Cost Function
Loss function
Error function

Examples in Regression

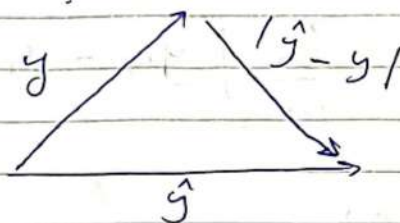
(MAE)

(MSE)



proof :

y	$\hat{y}$	$ \hat{y} - y $
$\vdots$	$\vdots$	$ e_1 $
$\vdots$	$\vdots$	$ e_2 $
$\vdots$	$\vdots$	$ e_3 $
$\vdots$	$\vdots$	$\vdots$



$$|\hat{y} - y| = |e_1| + |e_2| + |e_3| + \dots = L_1\text{-norm}$$

$$MAE = \frac{L_1\text{-norm}}{m} = \frac{\|\hat{y} - y\|_1}{m}$$

$$\|\hat{y} - y\|_2 = \sqrt{e_1^2 + e_2^2 + \dots}$$

$$\|\hat{y} - y\|_2^2 = e_1^2 + e_2^2 + \dots$$

$$MSE = \frac{\|\hat{y} - y\|_2^2}{m} = \frac{\hat{E} \cdot \hat{E}}{m}$$

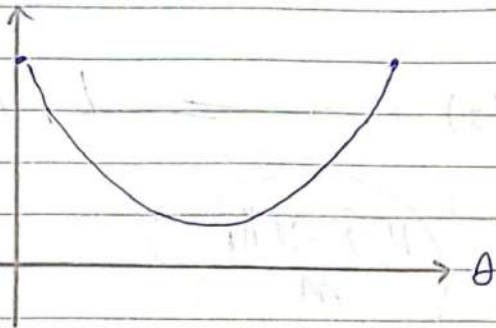
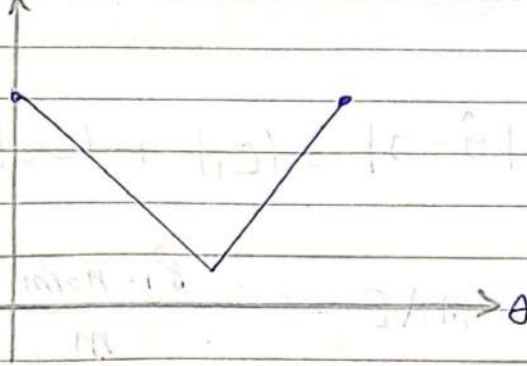


MAE

MSE

$$= \frac{1}{m} \sum_{i=1}^m |\hat{y}_{(i)} - y_{(i)}|$$

$$= \frac{1}{m} \sum_{i=1}^m (\hat{y}_{(i)} - y_{(i)})^2$$

 $J(\theta)$  $J(\theta)$ 

① أكثر سلاسة

① أقل سلاسة

② يهتز error

② مقاوم لـ outliers $()^2$

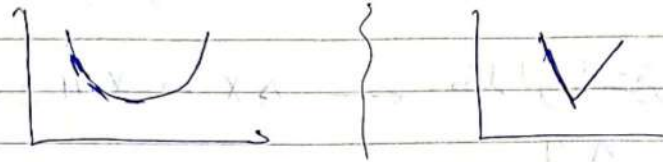
خط

في الـ (MAE) المستقيمة عند الـ min غير معرفة $((\frac{dJ}{d\theta} = \text{undefined}))$ لذلك لما نجد الـ min بنقرب منه فقط.

(curve)

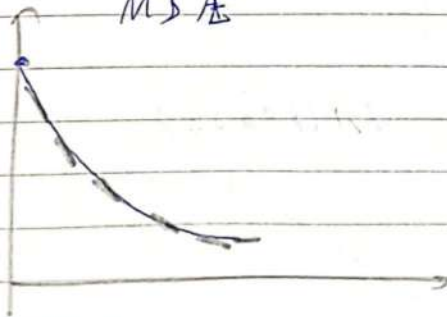
في حالة استخدام MSE قيمة $gradient$ بتقل $\downarrow$

بينما في حالة MSA بتفقد ثابتة $\Rightarrow$ (line)

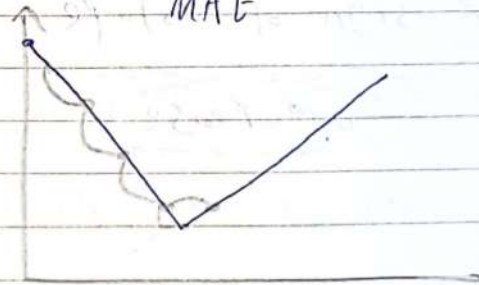


لذلك في حالة استخدام (MAE) يمكن حدوث overshoot

MSE



MAE



$\Rightarrow$ grad: decrease $\downarrow$ } grad $\Rightarrow$ const

(at min)

$$\frac{dJ}{d\theta} \approx 0$$

$$\frac{dJ}{d\theta} = \text{undefined}$$

so,

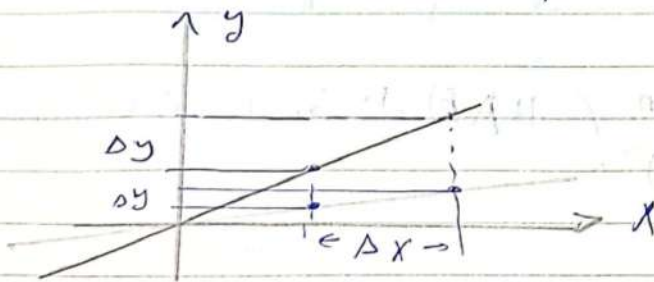
MS E $\rightarrow$ more smooth in training



④ Update θ :

↳ gradient → Amount of change sensitivity $\nearrow \nearrow$

لـ لو تغيرت في x بـ Δx ← مقدار التغير في y :



* sign of slope :

Decrease

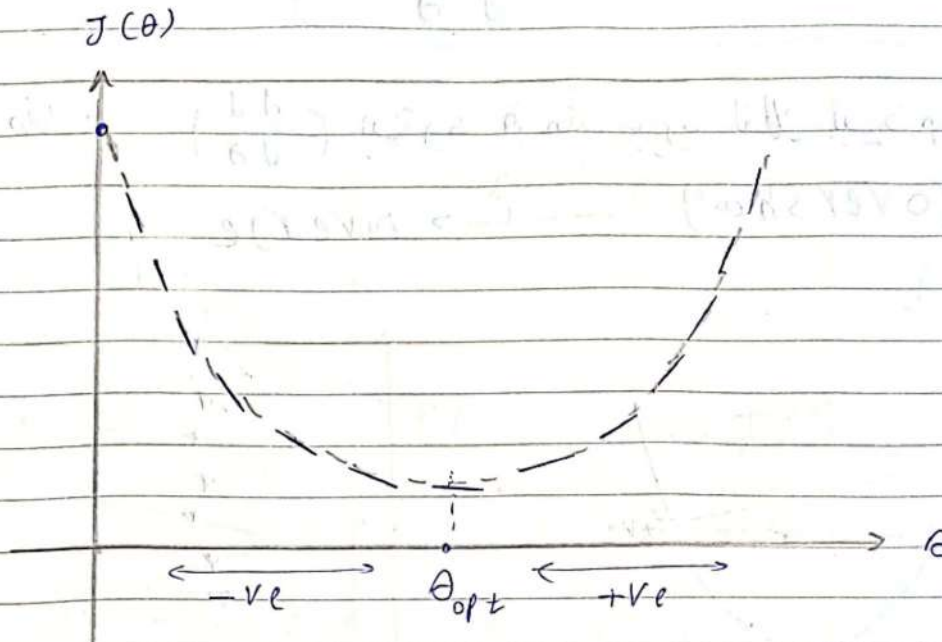
Increase

$x \uparrow \rightarrow y \downarrow$
(-ve)

$x \uparrow \rightarrow y \uparrow$
(+ve)



* what about Direction??



if gradient $\Rightarrow$ -ve

if gradient $\Rightarrow$ +ve

I need to increase θ

I need to decrease θ

$$\frac{dJ(\theta)}{d\theta} = -ve$$

$$\frac{dJ}{d\theta} \approx 0$$

$$\frac{dJ(\theta)}{d\theta} = +ve$$

$\theta \uparrow$

$\Downarrow$
(Stop)

$\theta \downarrow$

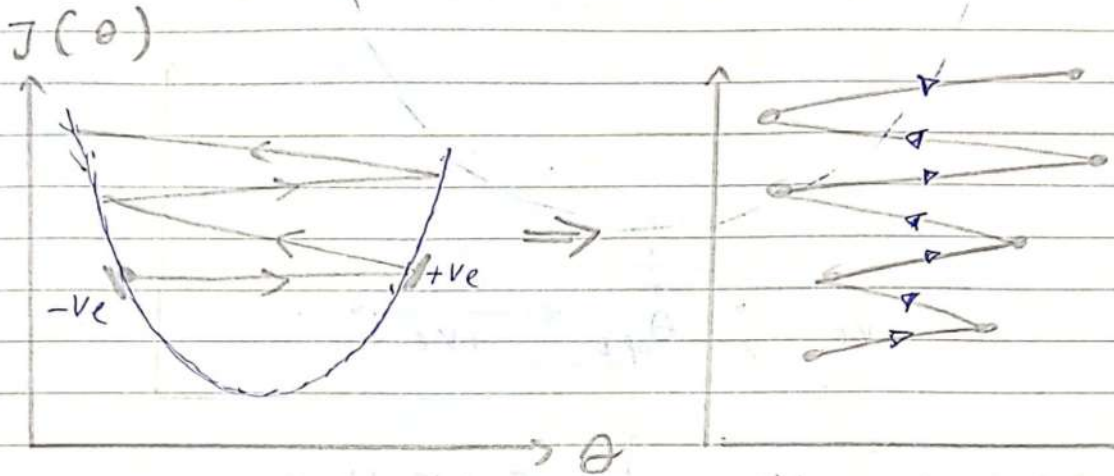
$\Rightarrow \boxed{\theta_{opt}}$

Direction of update $\theta \Rightarrow$ In the opposite Direction of gradient



$$\theta_{t+1} = \theta_t - \frac{dJ(\theta)}{d\theta}$$

لا يجب أن تكون قيمة مبررة لذلك يجب أن يكون $\left(\frac{dJ}{d\theta}\right)$ $\rightarrow$ Diverge (overshoot)



$\rightarrow$ over shot

$\rightarrow$ divergence oscillation

لذلك : م حريف (parameter) بحيث يقلل من

قيمة $\left(\frac{dJ}{d\theta}\right)$

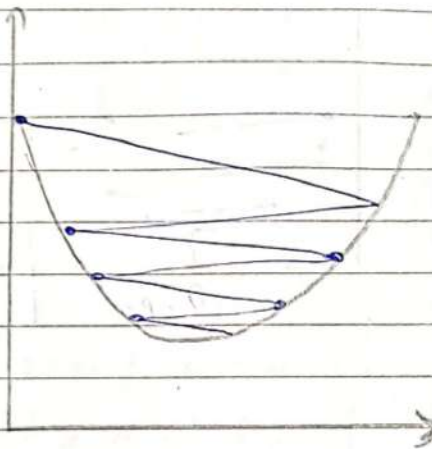


$$* \theta_{t+1} = \theta_t - \alpha \frac{dJ(\theta)}{d\theta}$$

(learning rate hyperparameter)

$$0 < \alpha < 1$$

let $(\alpha = 0.9)$



1) overshoot

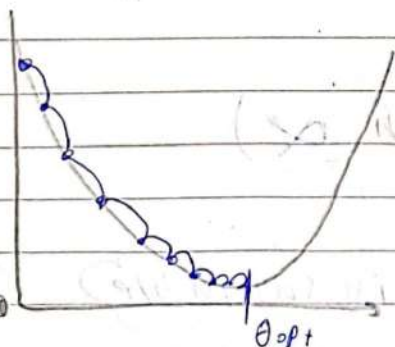
2) converge

هذه هي overshoot
ولكن بسبب convergence
هذه هي لا (min)

Iterations كثير

بسبب (oscillations)

let $\alpha = 0.1$



→ 1) No overshoot

→ 2) converge

ولكن بسبب انه لا قيمة (مغيرة)
منها عدد جيرة من
Iterations غلطان فوجد
لا (min)

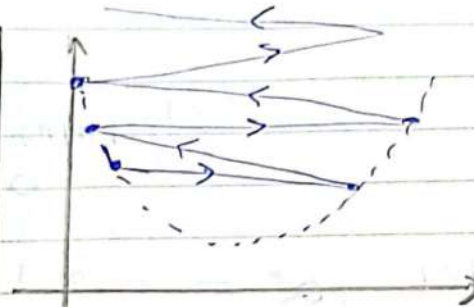


* learning rate effect :

1

Very large
(α)

- a) overshoot
b) Diverge

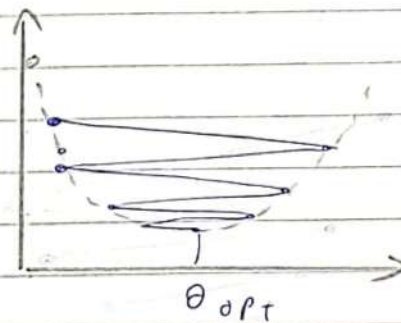


$\alpha = 1$

2

Large (α)

- a) overshoot
b) Converge



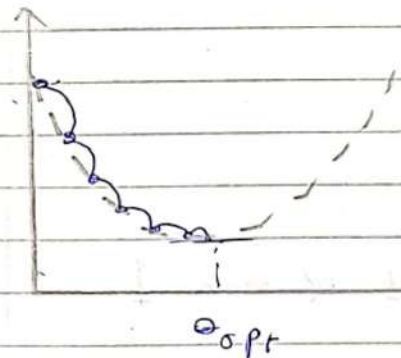
$\alpha \approx 1$

Iterations $\uparrow\uparrow$

3

Very small
(α)

- a) No overshoot
b) Converge



$\alpha \approx 0$

Iterations $\uparrow\uparrow$

We must choose $\rightarrow$ (optimum α)

- $\rightarrow$ No overshoot
- $\rightarrow$ No. of iterations is suitable



Note :

* Hyper Parameter.

لها هي القيم التي أنا بحددها قبل ما ار Training يبدأ

- ex:
- 1) learning rate
 - 2) Number of epochs
 - 3) Batch size
 - 4) Regularization parameters
 - 5) No. of (layers / units) per layer (NN)

Parameter

hyper parameter

learned automatically
(by model)

chosen manually

ex:

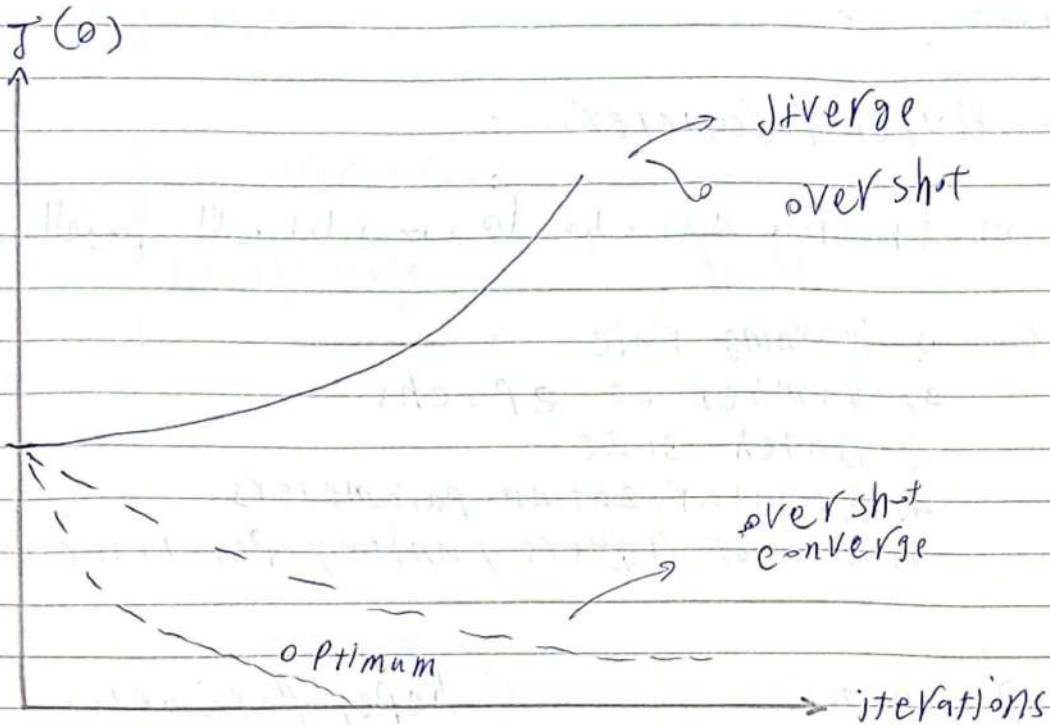
→ weight

↳ biases

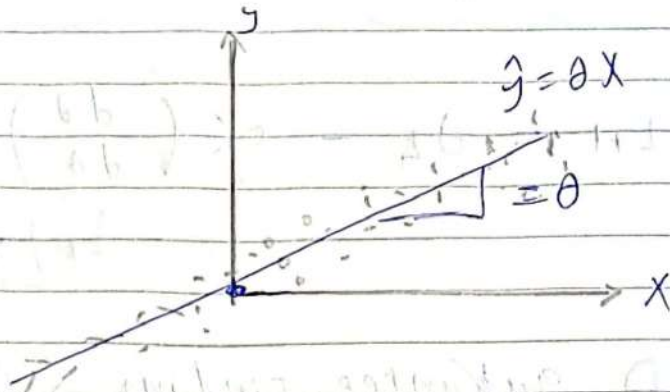
→ learning rate

↳ batch size





* example :



1 Initialize ($\theta = 0$)

2 $h_{\theta}(x) = \hat{y} = \theta x$

3 Evaluate (Cost Function)

or $\begin{cases} \text{MSE} \\ \text{MAE} \end{cases}$

4 Define $\begin{cases} \text{Direction of change} \\ \text{Amount} \end{cases}$

5 update $\theta \Rightarrow \theta_{t+1} = \theta_t + \text{Amount}$

$$\theta_{t+1} = \theta_t - \alpha \frac{dJ}{d\theta}$$



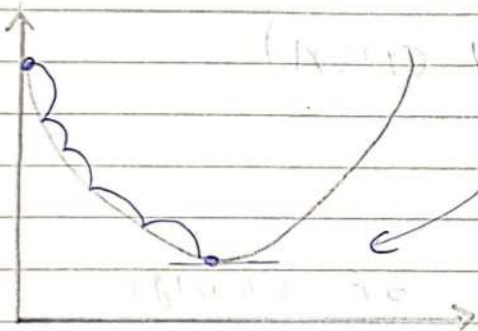
* Gradient Descent & Convexity :

$$\theta_{t+1} = \theta_t - \alpha \left(\frac{dJ}{d\theta} \right)_t$$

$$\nabla \theta_t$$

* GD guarantee finding $\rightarrow$ (local minimum)

i) Convex Function: $\rightarrow$ have one



Global minimum
= local

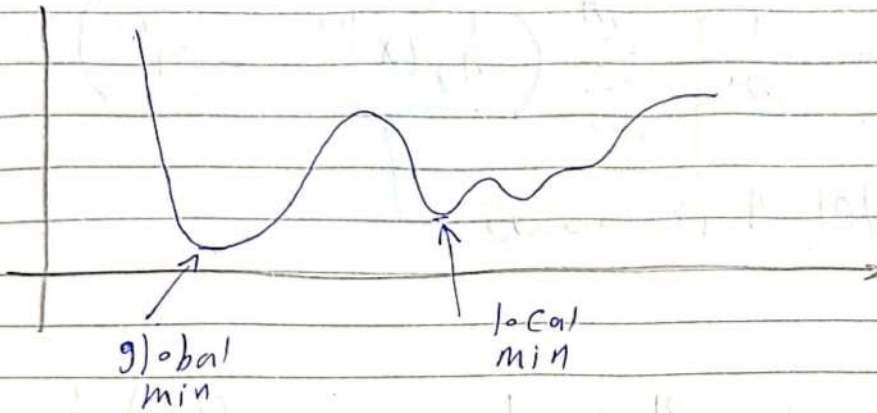
ال GD متأكد من إيجاد الحل الأمثل ودايماً في حوض
فاحية ال (min) . (U)

* خذ في الحسبان +
من أغلب ال problems في ال ML

عبارة عن (convex)



ii) Non-convex function



مثال GD ممكن يواجه مشكلة لأنه

ممكن يتجه ناحية

global min

local min

saddle point

* لا يقف *

مثال (NN) الدوال Non-convex

لذلك مثال معب الوصول إلى (global min)

و بفهم الدخول هارن زي

Adam

momentum



$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \left(\frac{h_{\theta}(x)^{(i)}}{1} - y^{(i)} \right)^2$$

ex:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \left(\frac{1}{1 + e^{-\theta^T x^{(i)}}} - y^{(i)} \right)^2$$

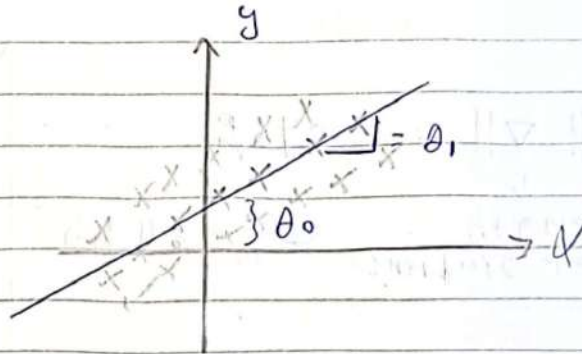
cross entropy loss function

convex قوس و محدب



I) Single - Variable Linear Regression

X	y	$\hat{y}$
x_1	y_1	$\hat{y}_1$
x_2	y_2	$\hat{y}_2$
x_3	y_3	$\hat{y}_3$
x_4	y_4	$\hat{y}_4$
x_5	y_5	$\hat{y}_5$



$$\hat{y} = \theta_0 + \theta_1 X$$

steps:

① Assume $\theta_0 = 0$, $\theta_1 = 0$, $\alpha = 1$

② $h_0(X) = \theta_0 + \theta_1 X$

③ Evaluate Error:

$$J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m \left(\underbrace{h_0(x^{(i)})}_{\theta_0 + \theta_1 x^{(i)}} - y^{(i)} \right)^2$$

④ Define: $\left(\frac{\partial J}{\partial \theta_0} \right)_t$, $\left(\frac{\partial J}{\partial \theta_1} \right)_t$

$$\theta_{0,t+1} = \theta_{0,t} - \alpha \left(\frac{\partial J}{\partial \theta_0} \right)_t$$

$$\theta_{1,t+1} = \theta_{1,t} - \alpha \left(\frac{\partial J}{\partial \theta_1} \right)_t$$

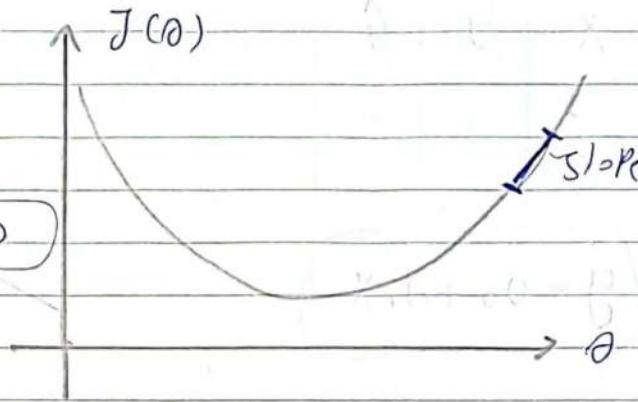


* Gradient Vector of single-Varial linear Reg.

$$\|\nabla\| = \text{slope}$$

length
of gradient

طول الـ slope



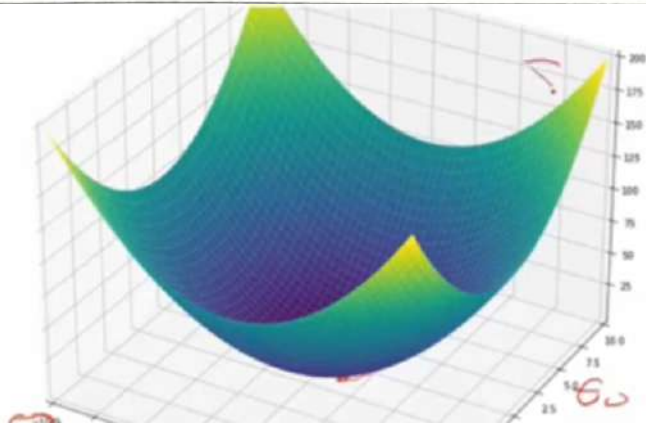
$$J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m [(\theta_0 + \theta_1 x^{(i)} - y^{(i)})^2]$$

assume: $J(\theta_0, \theta_1) = \theta_0^2 + \theta_1^2$

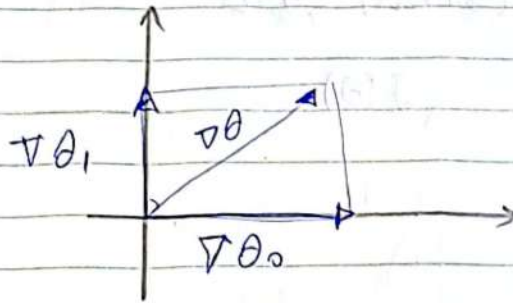
يعني الـ gradient يعبر عنه التغير في الـ (J)

لما أغير في θ_0 و θ_1

$J(\theta_0, \theta_1) \rightarrow$ multi-variable Function



$$\nabla \theta = \begin{bmatrix} \nabla \theta_0 \\ \nabla \theta_1 \end{bmatrix}$$



$$\therefore \nabla = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \end{bmatrix} \rightarrow \text{gradient vector}$$

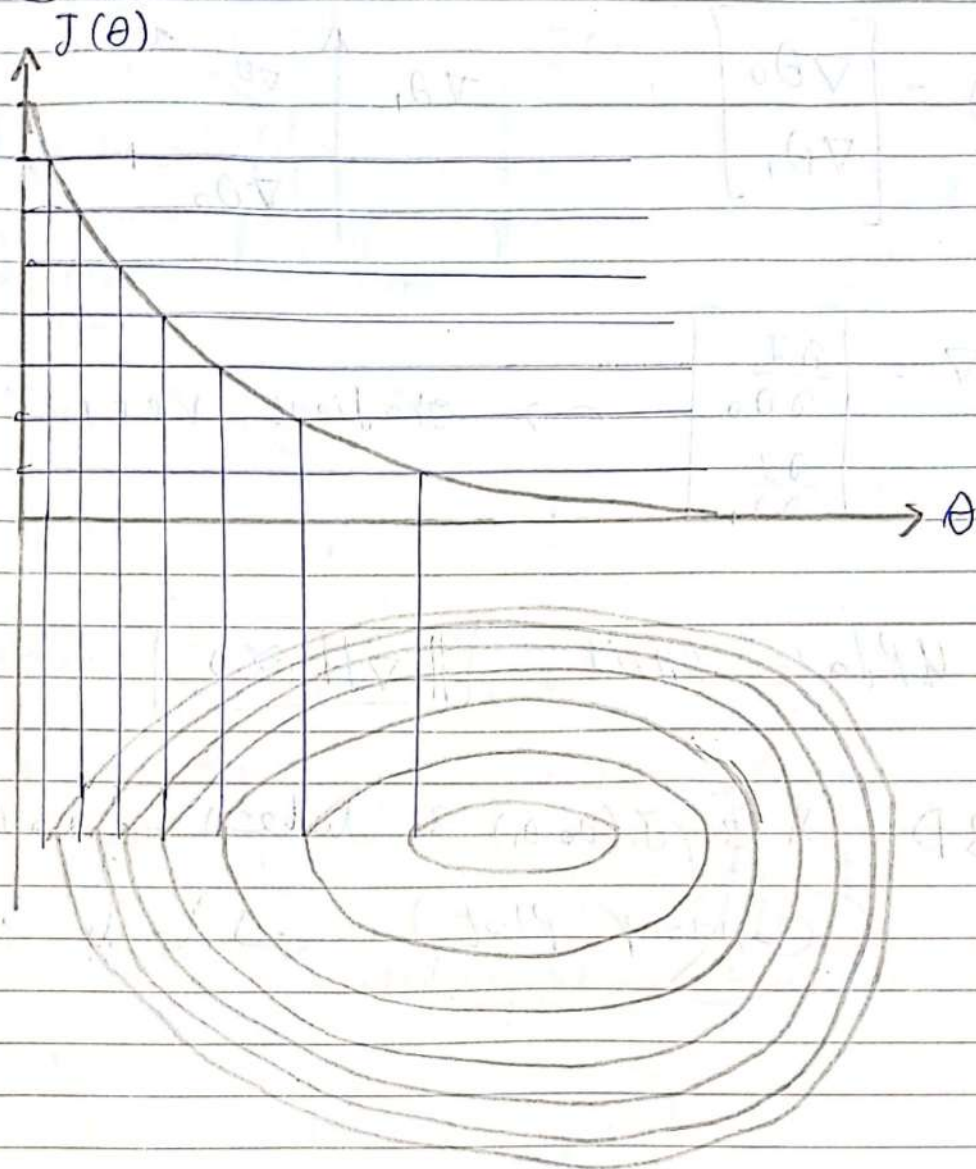
we update until : $\|\nabla\| \approx 0$

* فن الصعب التعامل مع $J(\theta_0, \theta_1)$ في الـ 3D

لذلك بدلًا من ذلك $\leftarrow$ contour plot



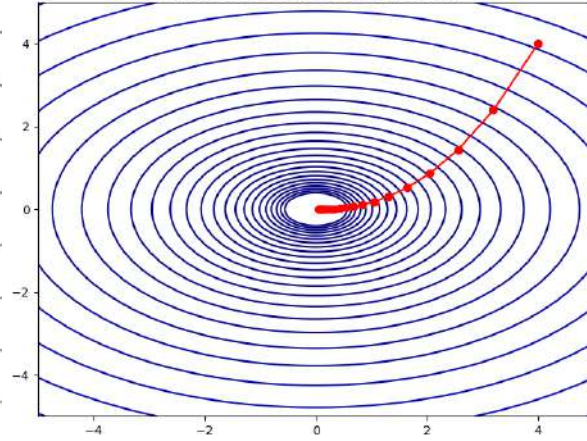
* contour plot :



wide circles $\rightarrow$ small gradient
narrow circles $\rightarrow$ large gradient



Graddient Descent on a Covex Function



مع نيات قيمة ال $(\nabla J(\theta))$

كل ما ينقرب ال $\min$

كل ما الدوائر سمك أو سع
كل ما قيمة ال θ هو هتقل

بعض الأول هكون أسرع θ أبعد

كل ما ينقرب ال $\min$ كل ما ال slope يقل
(إبطأ)



* Apply GD :

(1) assume θ_0, θ_1

(2) $h_0(x) = \theta_0 + \theta_1 x^{(i)}$

(3) $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (\underbrace{\hat{y}^{(i)}}_{h_0(x^{(i)})} - y)^2$
 $= \theta_0 + \theta_1 x^{(i)}$

(4) $\frac{\partial J}{\partial \theta_0}, \frac{\partial J}{\partial \theta_1}$ بحط (2) في المقام
 ولسان انتفاضل

* $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)})^2$

$\frac{\partial J}{\partial \theta_0} = \frac{2}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)}) \cdot 1$

$\frac{\partial J}{\partial \theta_1} = \frac{2}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)}) \cdot x^{(i)}$

$\nabla \theta = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \end{bmatrix}$

(5) $\theta_0, t+1 = \theta_0, t - \alpha \frac{\partial J}{\partial \theta_0} |_t$

$\theta_1, t+1 = \theta_1, t - \alpha \frac{\partial J}{\partial \theta_1} |_t$



* multi-variable

x_0	x_1	x_2	...	x_n	y	$\hat{y}$
1	.	.	.	.	.	.
1	(	(	(	.	.	.
1	.	.	.	.	.	.
:	.	.	.	.	.	.

$$h_0(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n \quad (\text{model})$$

$$\theta_{\text{vector}} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}, \quad X = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$$

: (Xa)

... علشان عدد ال (dim) بيت زي بـ ٧



$$\begin{array}{c}
 \xleftarrow{n+1} \xrightarrow{\quad} x_0 \\
 \begin{bmatrix} X \end{bmatrix} \begin{bmatrix} \theta \end{bmatrix} = \begin{bmatrix} Y \end{bmatrix} \\
 m \times (n+1) \quad (n+1) \times 1 \quad m \times 1
 \end{array}$$

Note:

$$\text{the g-gradient} \Rightarrow \nabla \theta = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \vdots \end{bmatrix}$$

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_0^{(i)} = 1$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_1^{(i)}$$

$$\frac{\partial J}{\partial \theta_2} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_2^{(i)}$$

$$\vdots$$

$$\frac{\partial J}{\partial \theta_{n+1}} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_{n+1}^{(i)}$$



* Feature Scaling :

لما يكون عندنا Features متفاوتة في Ranges

1 → 5 1000 → 2000

X_1	X_2	y

بيانات على مساحة في
predict

لذلك بحاجة إلى خلية Parameter

θ_1

θ_2

مغير

مغير

X_1

X_2

(مغيرة)

(مغيرة)

بحيث أن $(\hat{y})$ تعطى قيمة منطقية

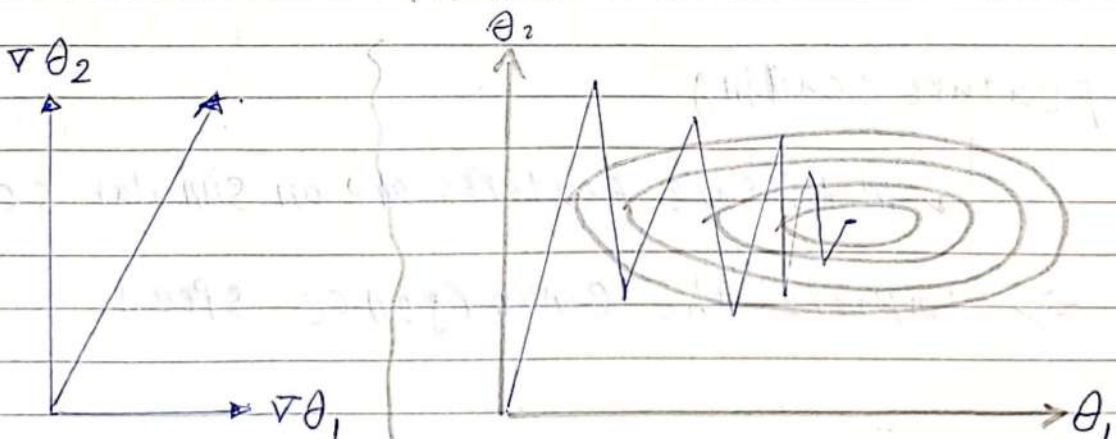
(Feature)

Parameter

gradient

• X is large $\uparrow \rightarrow \theta$ is small $\downarrow \rightarrow \nabla \theta$ is large $\uparrow$

• X is small $\downarrow \rightarrow \theta$ is large $\uparrow \rightarrow \nabla \theta$ is small $\downarrow$



النتيجة :
 ان يعمل $update$ في اتجاه θ بقيمة كبيرة
 زغ في اقل قيمتها صغيرة

ويعمل $update$ في اتجاه θ بقيمة صغيرة
 زغ في اقل قيمتها كبيرة

الحل : Feature Scaling

- 1) Min-max Normalization
- 2) Mean Normalization
- 3) Robust scaling
- 4) Unit-Vector Normalization

Feature scaling

↳ Make sure features are on similar scale

⇒ Improve the convergence speed.



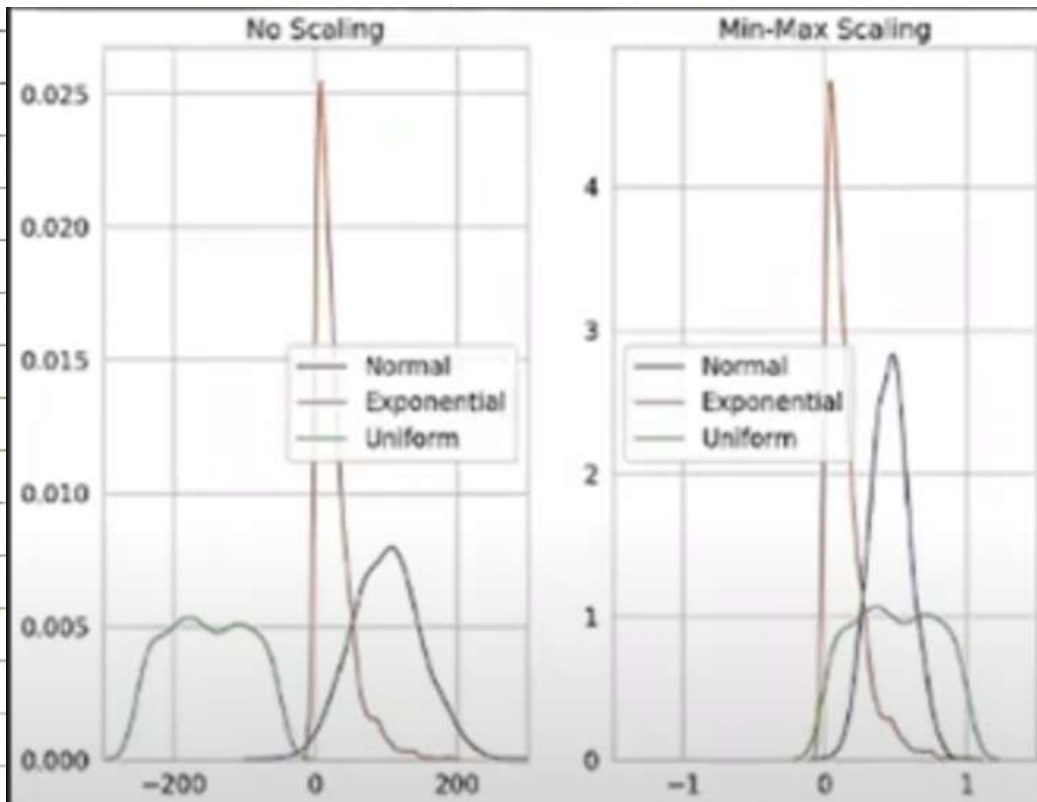
1. min-max normalization - $[0, 1]$

$$x' = \frac{x - \min}{\max - \min}$$

* It scales each feature in $[0, 1]$ range

* preserves the shape of the original distribution

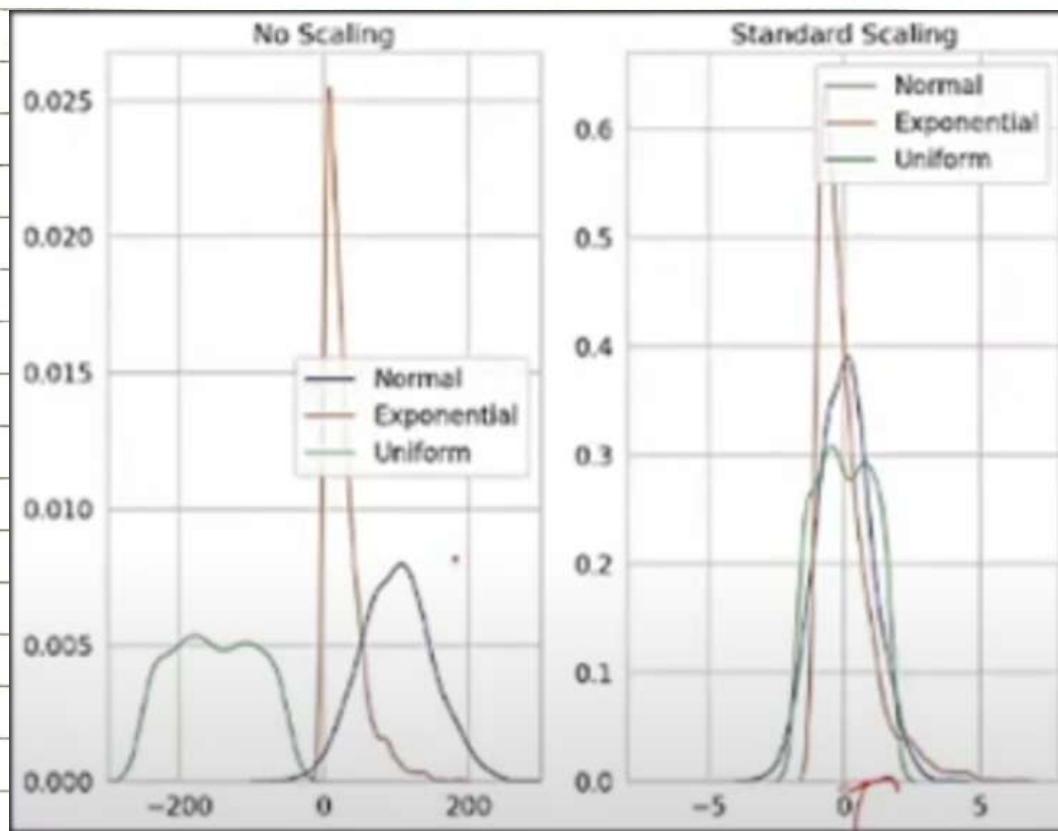
* sensitive to outliers.



2.1 Mean Normalization (Standardization)

⇒ produces a distribution centered at 0 with a standard deviation of 1

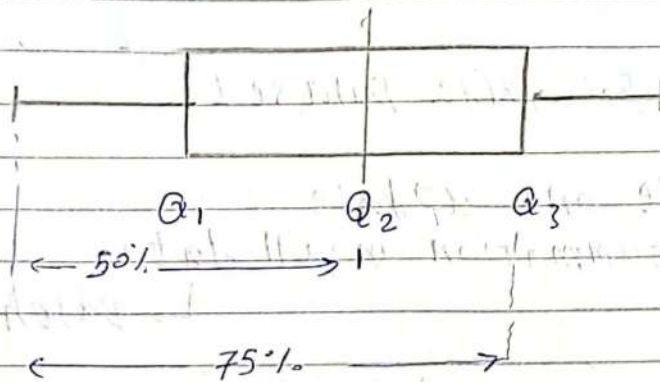
⇒ with outliers → the data will be scaled to a small interval.



3 Robust Scaling:

$$X' = \frac{X - \text{median}}{IQR} = \frac{X - Q_2}{Q_3 - Q_1}$$

↖ median



* Robust to outliers



* Gradient descent has some limitations:

↳ GD is a simpler optimizer and has some problems:

① stuck with Noisy / Non-convex surfaces

② Expensive for large datasets.

↳ To make one update

⇒ Do summation on all data

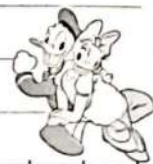
↳ Batch

* When use all data ⇒ overhead computing

$(X^T X)^{-1}$
may can't be performed

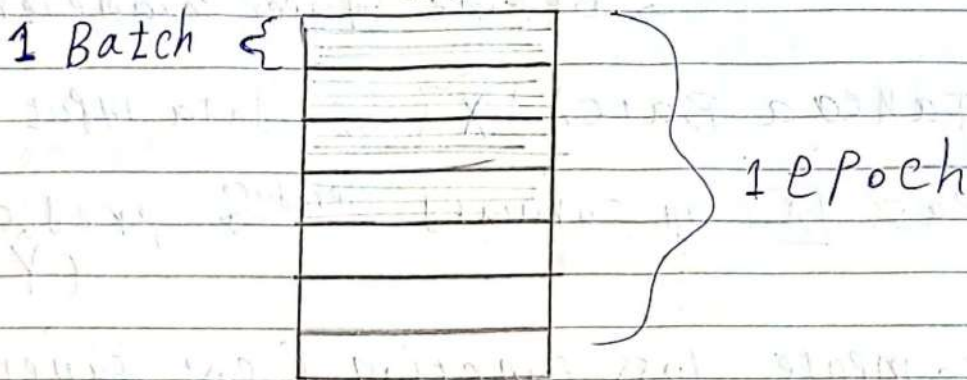
↳ Can't fit in memory

↳ long time

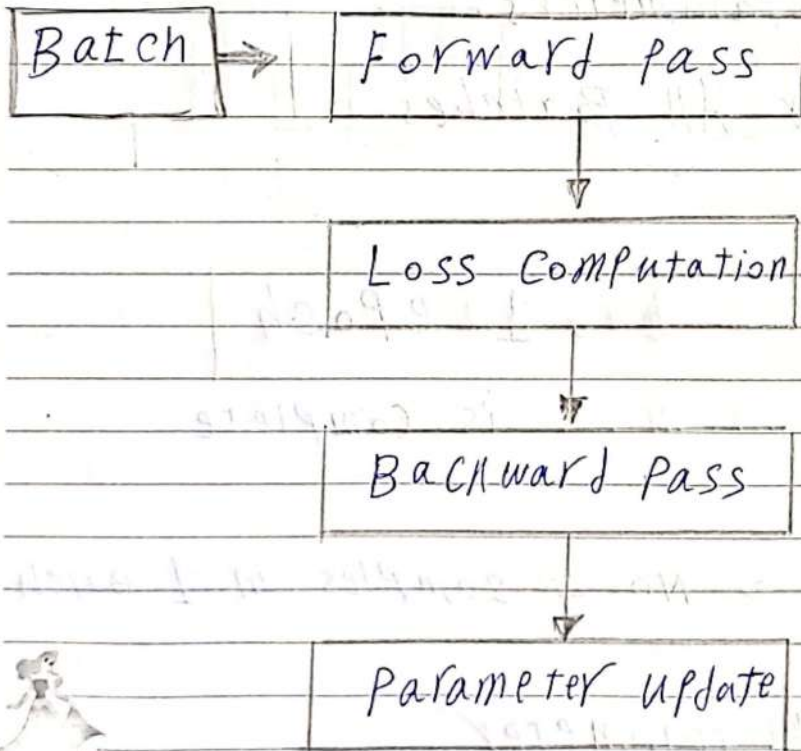


* Key Concepts :

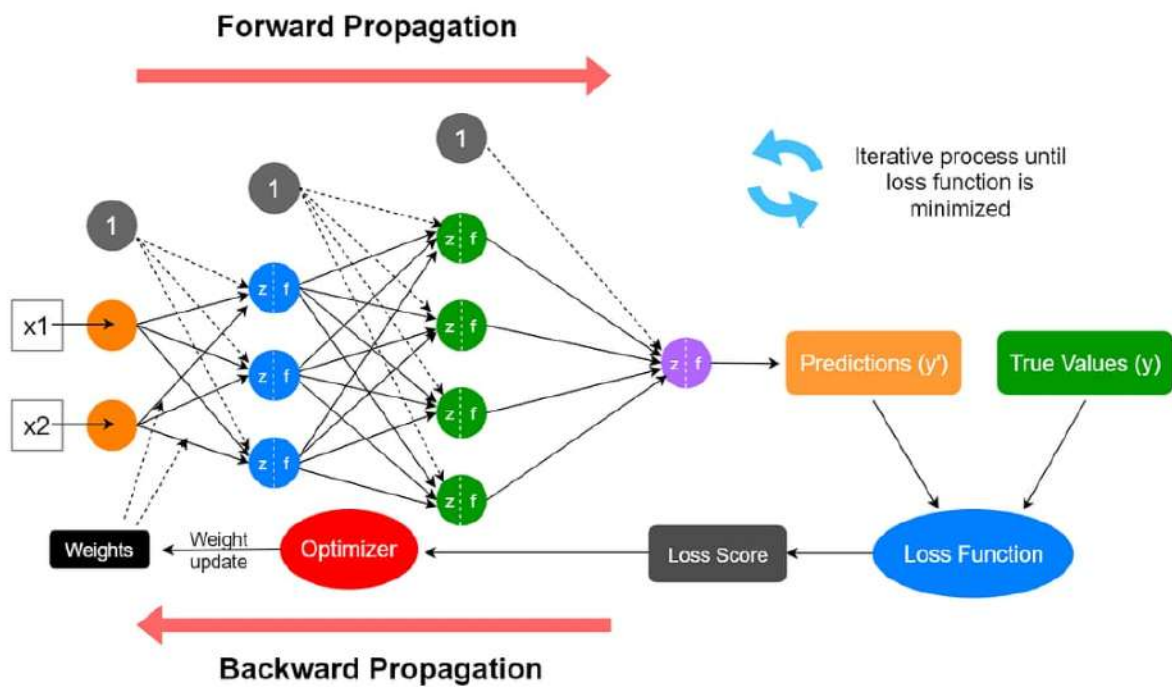
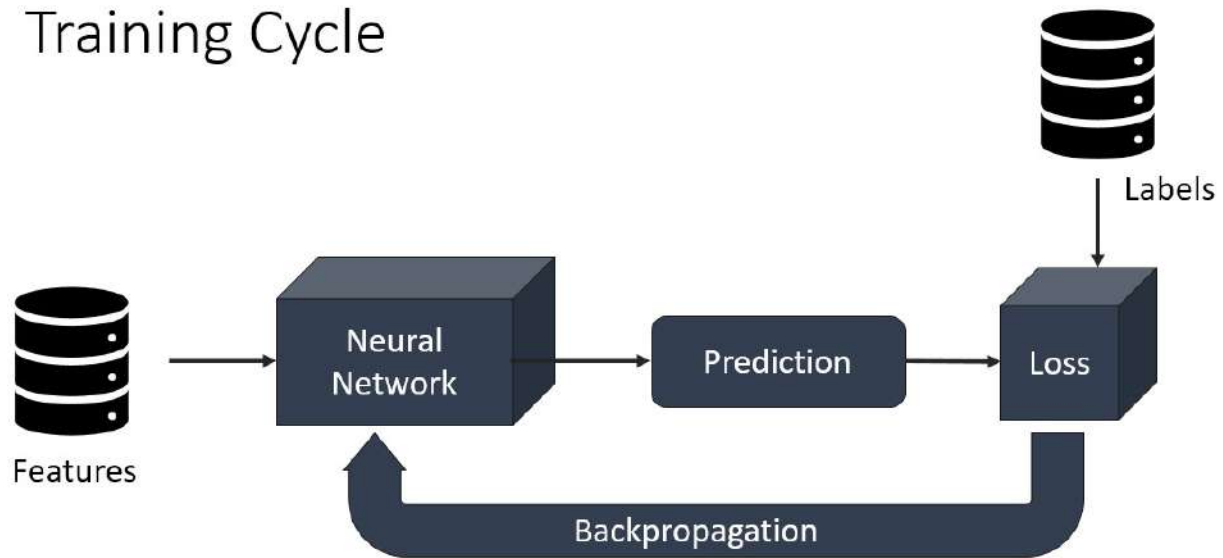
① Training cycle :



For: each Batch :



Training Cycle



* After Initialization

- start with random or pre-trained parameters
- Define Hyperparameters

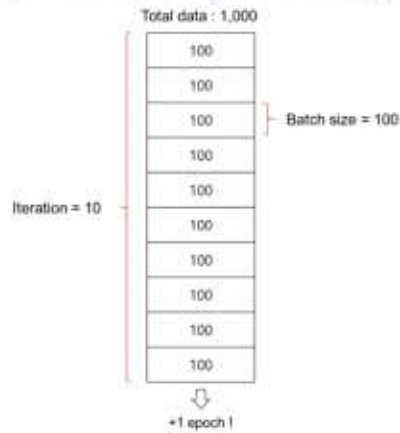
- ① Take a Batch X of data input
- ② Pass X in forward → produce predictions ($\hat{Y}$)
- ③ Compute loss function (cost function)
- ④ Take the backward pass (Back Propagation)
- ⑤ update the parameters
- ⑥ Repeat for All Batches

All Batches → 1 epoch
are done is complete

(Note) Batch size ⇒ No. of samples in 1 Batch
→ hyperparameter

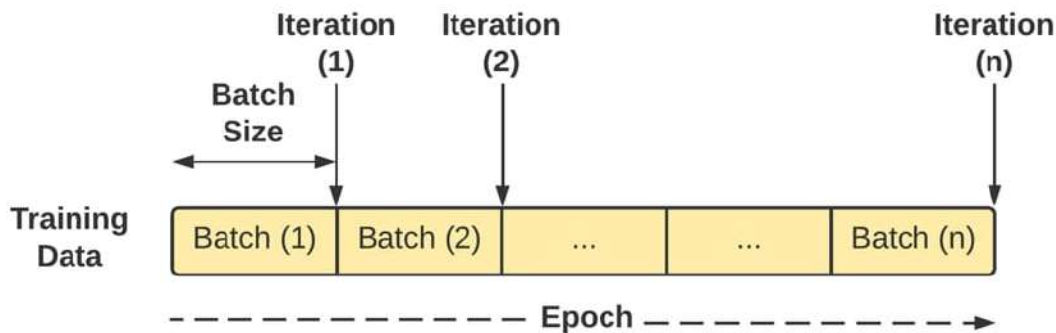


$$\text{No. of Batches} = \frac{\text{No. of samples}}{\text{Batch size}}$$



1 iteration → 1 Batch

$$\text{No. of Iterations} = \text{No. of Batches} \times \text{No. of Epochs}$$



1 Batch → needs → 1 iteration



* ① Batch GD (BGD) :

← علينا أن نحصل على single update
 ← محتاج إعداد على الداتا كلها
 ↳ $\text{Batch} = \text{all data}$

$$J() = \frac{1}{m} \sum_{i=1}^m [(h(x) - y^{(i)})^2]$$

1 epoch = 1 Batch = m

↳ 1 iteration

Data
sample 1
sample 2
i
i
sample m

→ Very slow with large amount of data

→ need huge memory



② Stochastic gradient descent (SGD)

← عمل single update
← (single observation) ج. ح. ت. ←

↳ [Batch = one sample = one observation]

$$J(\theta_0, \theta_1) = (\hat{y}^{(i)} - y^{(i)})^2$$

1 epoch → m observations (m Batches)

⇓
m iterations

* Disadvantages :

* loss curve → Very noisy

على البرقي إنها ممكن تكون موزة في ال (Non-convex)
← شئنا سم الخروج من ال local minima
بسبب ال (Noise)

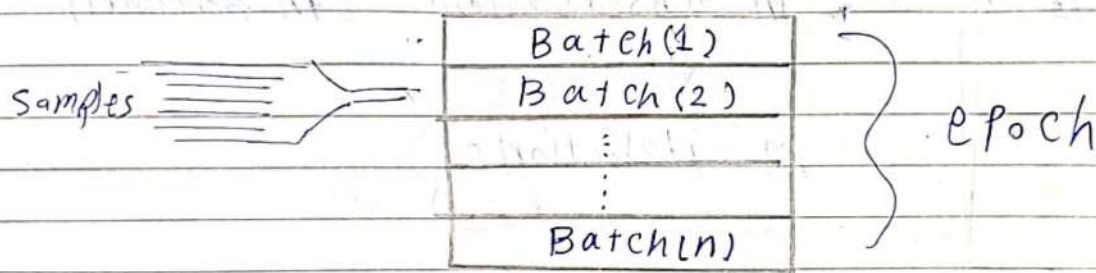


③ Mini-Batch Gradient Descent (MBGD)

له Rules أن تعمل single update
← تحتاج أنمي على جزء من الـ data في كل

→ Mini-Batch = subset of data

$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$$



المميزات:

- ① أسرع من BGD
- ② أقل تكلفة من SGD

العيوب:

له تحتاج أنحد الـ Batch size

→ hyperparameter



examples:

Data set has $\boxed{1000 \text{ samples}}$
we need perform $\boxed{10 \text{ epochs}}$

1) Using BGD:

→ Batch's size = 1000 sample

No. of Batches = 1

* No. of iterations = $1 \times 10 = \boxed{10 \text{ iterations}}$

2) Using SGD:

→ Batch's size = 1 sample

No. of Batches = 1000

No. of iterations = $1000 \times 10 = \boxed{10,000 \text{ iterations}}$

3) Using MBGD:

→ Batch's size = 100 (assumed)

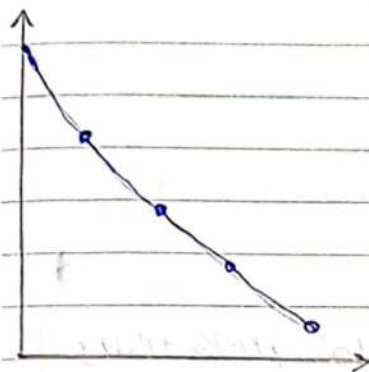
No. of Batches = 10

No. of iterations = $10 \times 10 = \boxed{100 \text{ iteration}}$

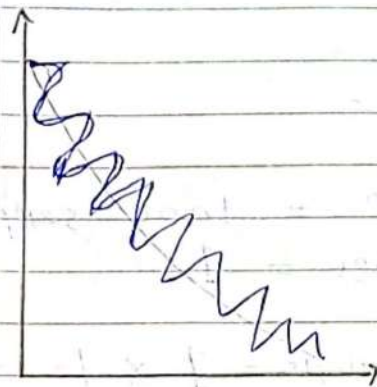


* BGD VS SGD VS MBGD :

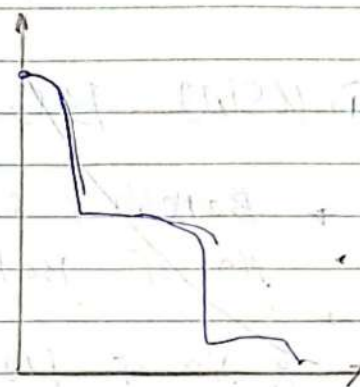
BGD



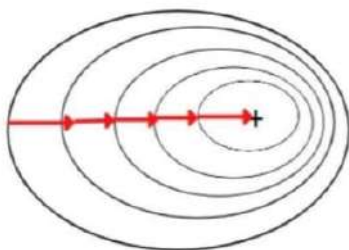
SGD



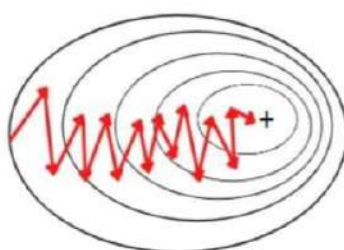
MBGD



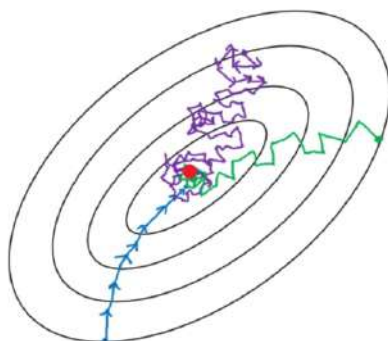
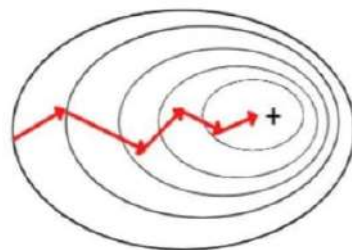
Batch Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent



— Batch gradient descent
— Mini-batch gradient Descent
— Stochastic gradient descent



* Random shuffle :

لإعادة ترتيب الـ data بشكل عشوائي في بداية كل epoch قبل تكوين Batches

السبب ؟

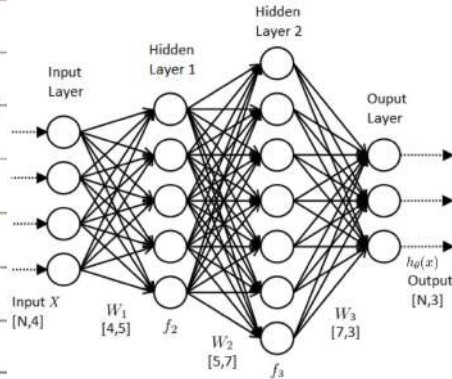
- ① تجنب الـ (Bias)
 - لو البيانات مرتبة مثلاً أول 500 عبارة عن قطع
 - وثاني 500 عبارة عن كلام الـ model هيستوف
 - في أول iteration قطع بس و Bias
- ② تحسين الـ (generalization)
 - الـ (model) لما يتعرفنا على epoch لتوزيع مختلف
 - هيقول من الـ (overfitting)
- ③ تسريع الوصول لـ (the Best minimum)
 - أحياناً الـ GD بيتقادي الـ local minimum

(Note) We perform Random shuffling

With → 1) SGD
→ 2) MBGB



* في ال (PL) بنسبة من (NN) :



$$\phi(X W^{[1]} + b) \Rightarrow A^{[1]} \rightarrow (\text{o/p of layer 1})$$

$$\phi(A^{[1]} W^{[2]} + b^{[2]}) \Rightarrow A^{[2]}$$

} { " (" (

علشان آ حسب ال (cost function) $\left(\frac{dJ}{dw}\right)$

بحسب آ عمل خطوة زيا دقة هذا ال ML وهي ال Backpropagation عن طريق ال Chain Rule

$$\frac{dJ}{dw} = \frac{dJ}{dA} \cdot \frac{dA}{dz} \cdot \frac{dz}{dw}$$

$$w_{t+1} = w_t - \alpha \left(\frac{dJ}{dw}\right)$$



* لما بحسب الـ $\frac{dJ}{dw}$ بهرب في أكثر من قيمة

$$\left(\frac{dJ}{dA} \right) \cdot \left(\frac{dA}{dz} \right) \cdot \left(\frac{dz}{dw} \right) \text{ أقل من واحد}$$

$< 1 \quad < 1 \quad < 1$

وبالتالي قيمة الـ $\left(\frac{dJ}{dw} \right)$ هتقل أكثر

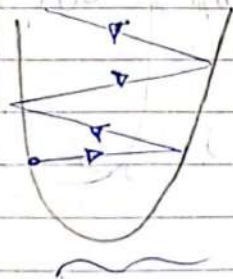
كل ما أبعد أكثر ← كل ما تقل أكثر
(زيادة عدد الـ layers)

Problems in DL: وبشكل كذا مشكلة:

① Vanishing Gradient ①

⇒ gradient → very small ≈ 0

② exploding gradient (if $\bar{I}$ increase $\propto$)

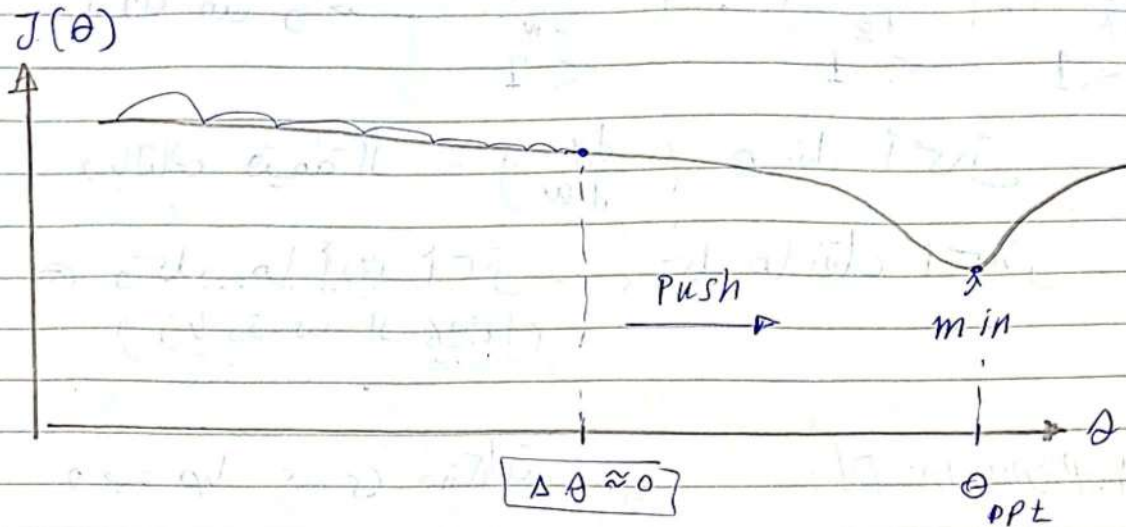


الحل ←

Momentum Based GD



* Momentum Based GD :



لاحظ هنا اننا مضى (push) على ان اقترأ وحل
 لـ (θ_{opt})

لأننا ان $\boxed{grad = 0}$ و accuracy قليلة

ورده معنا انه وحل (Vanishing grad.)

وحسن معنا انه وحل لـ (min)

والله بيحدر هو ان (performance)



* update equation:

$$\theta_{t+1} = \theta_t - \alpha \nabla \theta_t$$

$$\theta_{t+1} = \theta_t - [\alpha \nabla \theta_t + \gamma V_{t-1}]$$

لا أزود القيمة (γV_{t-1}) لـ (grad.) بحافظا على
 الـ gradien من انه يـ vanishing

$$V_t = \gamma V_{t-1} + \alpha \nabla \theta_t$$

$$\theta_{t+1} = \theta_t - V_t$$

$\gamma \rightarrow$ momentum hyperparameter ($\gamma < 1$)



Calculation:

assume: $\theta_0 = \theta_1 = 0$

$$V_0 = 0$$

$$V_1 = \gamma V_0 + \alpha \nabla \theta_1$$

$$= \alpha \nabla \theta_1$$

$$V_2 = \gamma [\alpha \nabla \theta_1] + \alpha \nabla \theta_2$$

$$V_3 = \gamma [\gamma \alpha \nabla \theta_1] + \gamma [\alpha \nabla \theta_2] + \alpha \nabla \theta_3$$

$$= \gamma^2 [\alpha \nabla \theta_1] + \gamma [\alpha \nabla \theta_2] + \alpha \nabla \theta_3$$

$$V_4 = \gamma^3 \alpha \nabla \theta_1 + \gamma^2 \alpha \nabla \theta_2 + \gamma \alpha \nabla \theta_3 + \alpha \nabla \theta_4$$

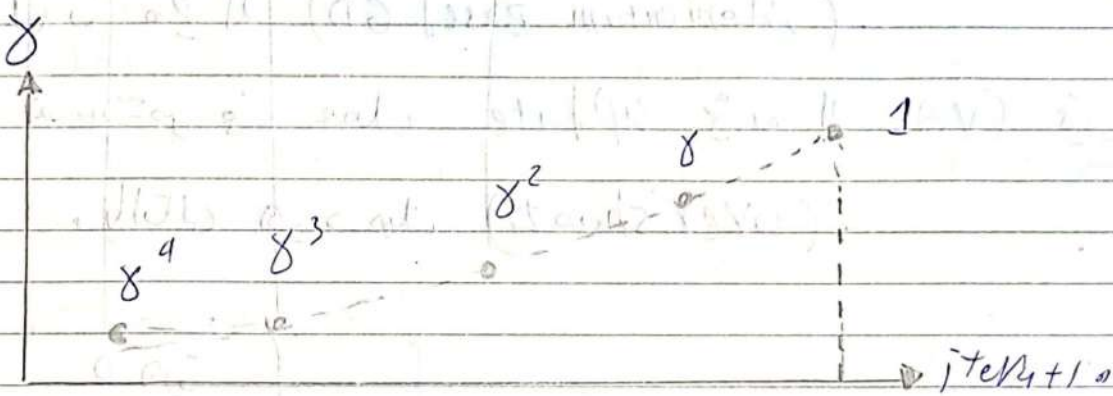
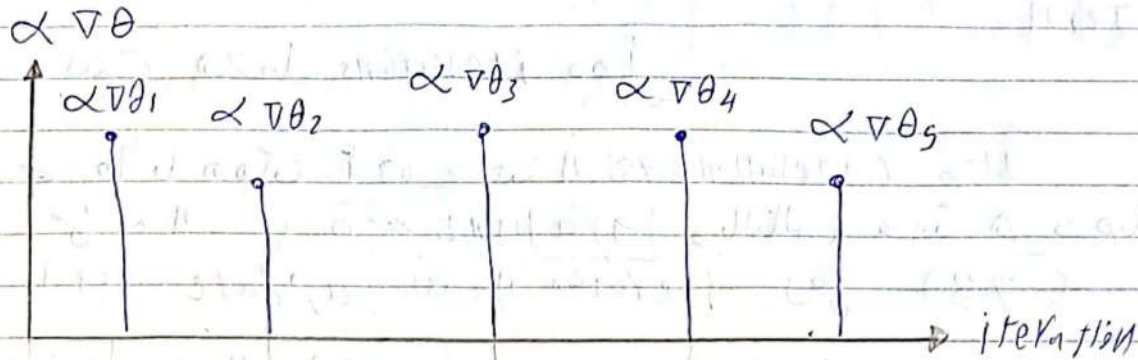
$$V_5 = \gamma^4 \alpha \nabla \theta_1 + \gamma^3 \alpha \nabla \theta_2 + \gamma^2 \alpha \nabla \theta_3 + \gamma \alpha \nabla \theta_4 + \alpha \nabla \theta_5$$

* كل ما تحصل iterations أكثر
كل ما أبعد عن النقطة الى كنت عندها
وبالتالي تأثير الـ history يكون أضعف

⇒ النقطة الأقرب ← الأقوى تأثيراً



* Exponential weighted sum:



Recursive Behavior

بين كل نقطة جوابا معلومة كل النقاط السابقة

لو قيمة الـ γ كانت عالية و γ حلت Vanishing

هو دحل (Divergence)

لذلك بنسخدم الـ (Technique) دة فة
ما يكون عنى (Vanishing)



مسألة الـ (Momentum Based GD) :

← يعني لو بسخدم (GD) بدون (Momentum Term)

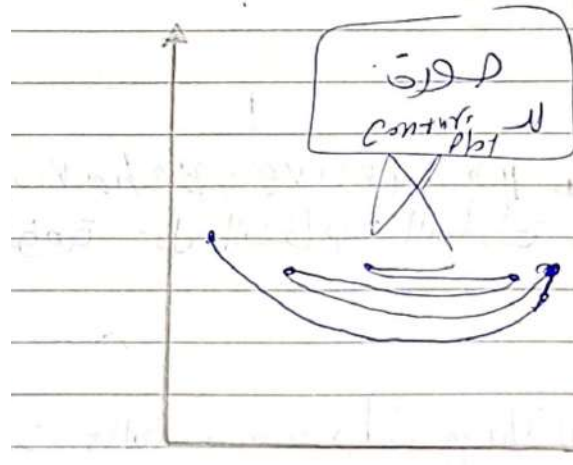
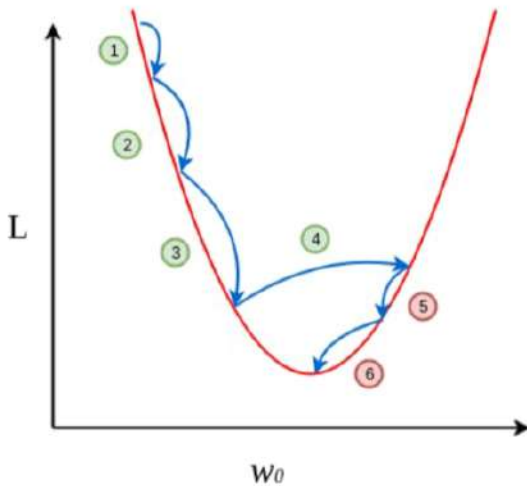
وكنت بعمل 100 iterations

← فآنا ممكن أقف عند الـ (iteration = 78) مثلاً
 $\nabla L \approx 0$ وبالتالي وبتالي وبتالي
 أي update بعد الـ iteration رقم (78).

لأن مع الـ (Momentum Based GD).

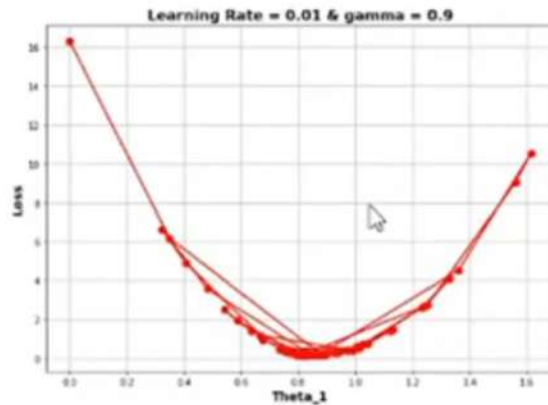
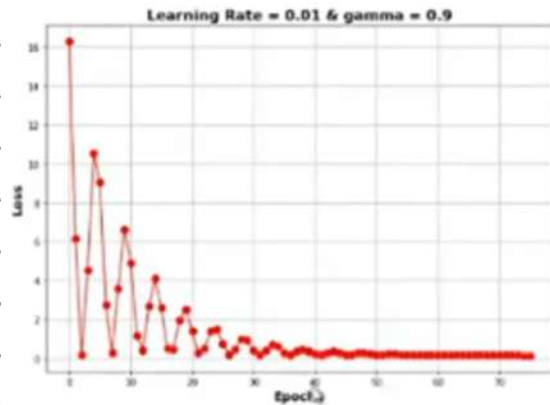
هيسمتر في عمل update عن الـ (∇L) في إشارة

وبالتالي هيدخل (overshoot).



* رغم انه ار oscillations كثير، ولكن

هو افضل من لا GD



→ Note

No technique is the best in general

* it based on
{

 experience
error & try

ولحل مشكلة ال oscillation

Nesterov Accelerated grad ←
(NAG)



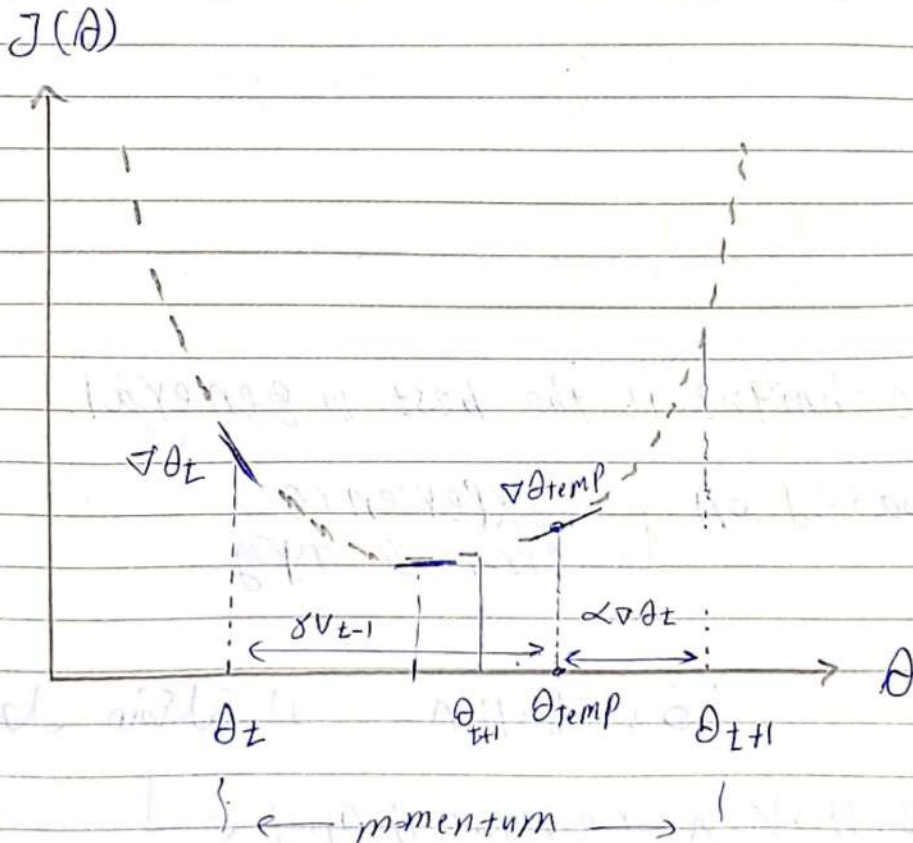
* Nesterov Accelerated Gradient (NAG)

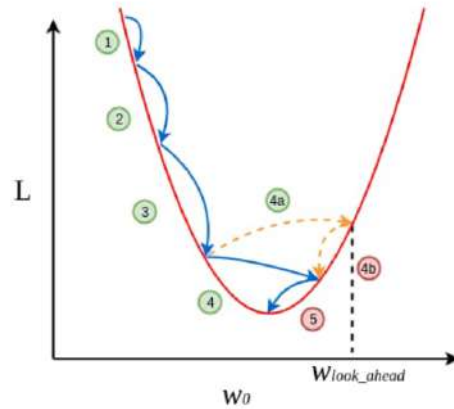
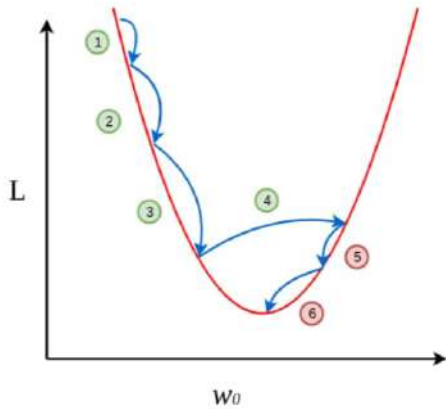
له طريقة لحل مشكلة الـ oscillations حول الـ min
في الـ (Momentum Based GD)

الفكرة :

→ بنحسب الـ (gradient) لننتقل إلى رحلتها
من الـ إلى كنت واقف عندها
ثم نعدل (update)

وبالتالي من الـ هو الـ overshoot) كبير





steps:

$$\theta_{temp} = \theta_t - \gamma U_{t-1}$$

$$\theta_{t+1} = \theta_{temp} - \alpha \nabla \theta_{temp}$$

$$U_t = \gamma U_{t-1} + \alpha \nabla \theta_{temp}$$

(Note)

$$\nabla \theta_{temp} > \nabla \theta_t$$

$$V_t^{moment} > V_t^{NAG}$$

$$\theta_t^{moment} > \theta_t^{NAG}$$



* لها كلفة بمابق ال (Feature scaling)

كان كل سانه الاختلاف في ال (Ranges)

لكن فيه مشكلة ثانية وهي Sparse Feature

No. of
bedrooms

No. of
pools

X_1

X_2

3

0

4

0

2

0

1

1

3

2

1

1

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

هنا من هقدر أ حل

المشكلة بـ Normalization

θ_1

θ_2

Dense
Feature

Sparse
Feature

محتاجين عدد كبير جداً
من ال (iterations)

(8th)

لـ θ_1 update في θ_2

θ_1

θ_2

θ_2



(الحل)

⇒ bigger weight $\rightarrow$ For $\nabla \theta_2$
 smaller $\rightarrow$ For $\nabla \theta_1$

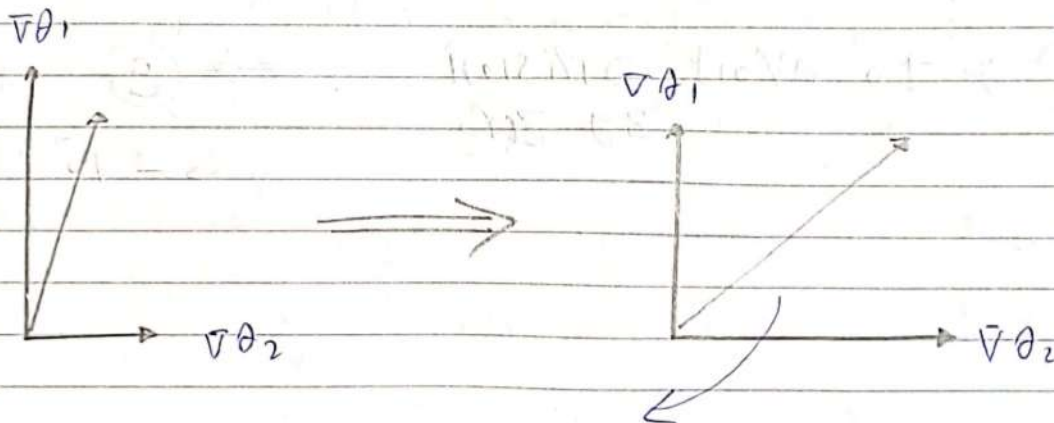
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\boxed{}} \nabla \theta_t$$

* Dense feature $\xrightarrow{\text{need}}$ ($\nabla \theta$ large)

$\boxed{} \rightarrow$ small

* Sparse feature $\xrightarrow{\text{need}}$ ($\nabla \theta$ small)

$\boxed{} \rightarrow$ large



Adaptive
gradient



* Adaptive gradient (Ada grad):

$$* V_t = V_{t-1} + (\nabla \theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{V_t} + \epsilon} \nabla \theta_t$$

* ϵ هنا مضاف بال magnitude لـ $(\nabla \theta_t)$ من أجل الأسيارة

$$\sqrt{V_t} \leftarrow \sqrt{V_t} + \epsilon \quad (\nabla \theta_t)^2$$

$\epsilon \rightarrow$ To avoid Division By zero

$$\epsilon \rightarrow \epsilon = 10^{-8}$$



Note

$$V_0 = 0$$

$$V_1 = (\nabla \theta_1)^2$$

$$V_2 = (\nabla \theta_1)^2 + (\nabla \theta_2)^2$$

$$V_3 = (\nabla \theta_1)^2 + (\nabla \theta_2)^2 + (\nabla \theta_3)^2$$

$$V_{n+1} =$$

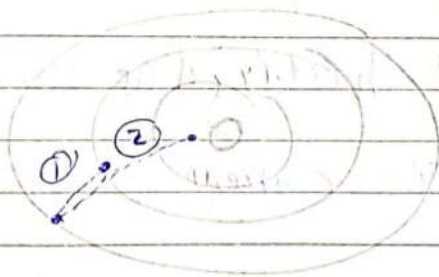
1

* إذا تى بجمع $(+)$ $\Leftarrow$ ال $V_{\pm}$ بتزيد

* ال $V_{\pm}$ بتزيد $\Rightarrow$ ال $V_{\pm}$ بتزيد $\Rightarrow$ ال $V_{\pm}$ بتزيد $\Rightarrow$ ال $V_{\pm}$ بتزيد

learning rate aggressive decay

ل تأثيره



① بدعمل (stop) فبأقرب ال min بكثر

أو

② بد stop قبل ال (min) بحاجة بزيادة

(قريب من min)



أدخل :

$$V_t = \square V_{t-1} + \square (v_{\theta t})^2$$

weight

< 1

history

current

weight

< 1

يحتوي أخذ جزء من

بالتالي V_t يقل ← تأثير (aggressive decay)
يقل

$$V_t = B V_{t-1} + (1-B) (v_{\theta t})^2$$

$$B < 1$$

* $B \uparrow \rightarrow$ أكثر history

* $B \downarrow \rightarrow$ أكثر current //



Note:

$$V_0 = 0$$

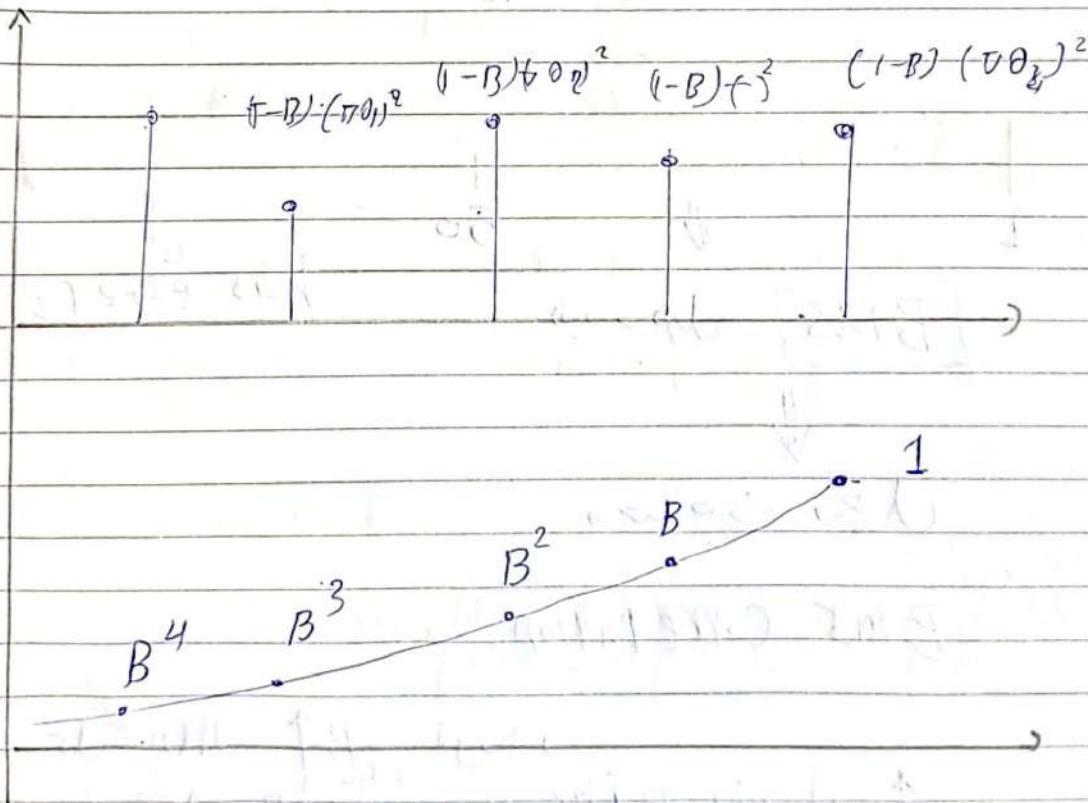
$$V_1 = B \cdot V_0 + (1-B)(\sigma\theta_1)^2$$

$$\therefore B(1-B)(\sigma\theta_1)^2 + (1-B)(\sigma\theta_2)^2$$

$$V_2 = B^2(1-B)(\sigma\theta_1)^2 + B(1-B)(\sigma\theta_2)^2 + (1-B)(\sigma\theta_3)^2$$

$$V_3 = B^3(1-B)(\sigma\theta_1)^2 + B^2(1-B)(\sigma\theta_2)^2 + B(1-B)(\sigma\theta_3)^2 + (1-B)(\sigma\theta_4)^2$$

→ EWMA



* From some math proof:

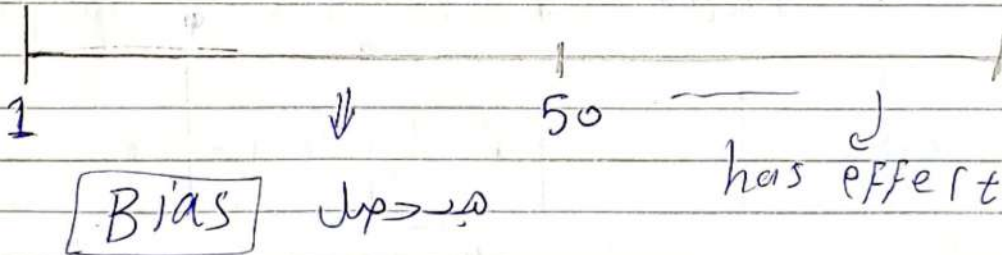
$$\frac{1}{1-B} = \text{No. of iterations}$$

عدد التكرار / observation التي تحتاجها

كلما كان B قريباً من 1 تأثيره

ex:

$$B = 0.98 \rightarrow \frac{1}{1-0.98} = 50 \text{ iteration}$$



محتاجين نعمل

Bias correction

كل ما $B \uparrow$ تزداد
كل ما تأثير B يزداد $\uparrow$



* Root Mean Square Propagation: (RMS Prop)

$$v_t = \beta v_{t-1} + (1-\beta) (\nabla \theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{v_t} + \epsilon} \nabla \theta_t$$

$$\beta = 0.9$$

$$\beta = 0.98$$

قيمة (B) الشهيرة



* كل طريقة مشكلة :-

Adagrad & RMSprop



في بعض الأحيان يتعامل مع
(Vanishing grad)

Momentum & NAG



في بعض الأحيان يتعامل مع
(Parsing Features)

السبب

$$v_t = B v_{t-1} + (1-B) (\nabla \theta_t)^2 \quad m_t = \gamma m_{t-1} + \delta \nabla \theta_t$$

مهتم بالقيمة (Dir) magnitude مهم

مهتم بالقيمة (Dir) magnitude مهم

ADAM



* Adaptive Moment Estimation (Adam)

$$m_t = B_1 m_{t-1} + (1 - B_1) \nabla \theta_t$$

$$v_t = B_2 v_{t-1} + (1 - B_2) (\nabla \theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{v_t} + \epsilon} m_t$$

oscillation $\Leftarrow$ Momentum

+

* يد مع بي

grad $\Leftarrow$ RMSProp يتكيف مع دج كل grad



كـ

لو شريت القام $\sqrt{v_t} + \epsilon$ $\Leftarrow$ (momentum)

لو استبدلت m_t بـ $\nabla \theta_t$ $\Leftarrow$ (RMSProp)



* Real life parameters :

$$B_1 = 0.9$$

$$B_2 = 0.999$$

$$\alpha = 0.0001 \rightarrow 0.0001$$

$$\epsilon = 10^{-8}$$

Adam $\rightarrow$ Mini-Batch $\rightarrow$ (DL) NN

هذا ليس مستخدم مثلاً (NAG)

لأنه تأثير ال overshooting مستقر وقوي جداً
 في بعض الحالات $\sqrt{V_{\theta}} + \epsilon$

ولكن

لذلك يمكن استخدام (NAG) على

$\rightarrow$ [NAdam] or [Adan]

أقل استخداماً



* Bias Correction

$$\hat{m}_t = \frac{m_t}{1 - B^t}$$

$$\hat{V}_t = \frac{V_t}{1 - B^t}$$

انا. ليس جزء ال $\hat{m}_t$ $\hat{V}_t$ في الحساب الى بعد

#

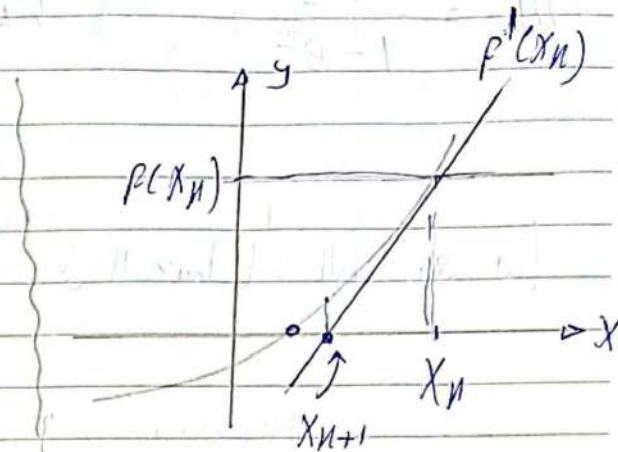


* GD connection to Taylor series :

$$f(x_{n+1}) = f(x_n) + f'(x_n)(x_{n+1} - x_n)$$

Objective :

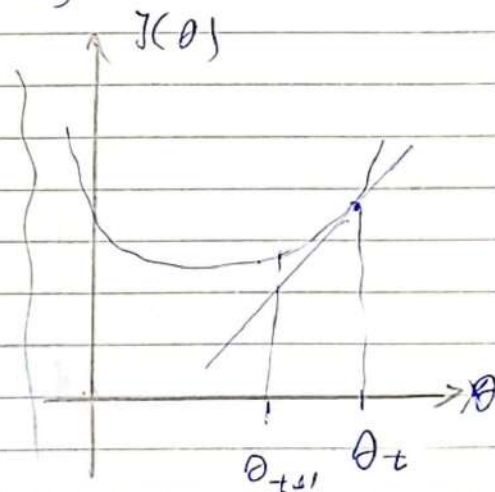
$$f(x_{n+1}) = 0$$



$$0 = f(x_n) + f'(x_n)(x_{n+1} - x_n)$$

$$\Delta x = x_{n+1} - x_n = -\frac{f(x_n)}{f'(x_n)}$$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



Target $\Rightarrow J(\theta_{t+1}) < J(\theta_t)$



$$f(x_{n+1}) = f(x_n) + \underbrace{f'(x) (x_{n+1} - x_n)}_{\rightarrow -ve}$$

من مميزات الـ (GD) انه يضمن انى دايماً
في اتجاه التناقص (↓).

$$f'(x) (x_{n+1} - x_n) = -ve$$

↓ ↓

$$\nabla(\theta_t) \cdot \Delta\theta_t = -ve$$

↑ dot product

يعني لازم الزاوية بينهم $< 90^\circ$.

$(-\Delta\theta_t)$ in the opposite direction of $(\nabla\theta_t)$.

$$\Delta\theta_t = -\nabla\theta_t$$

$$\theta_{t+1} - \theta_t = -\nabla\theta_t$$

$$\theta_{t+1} = \theta_t - \boxed{\quad} \nabla\theta_t$$

↓
(α)



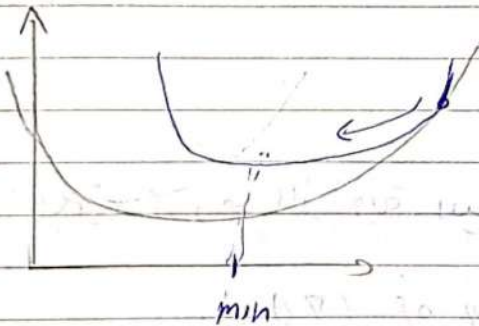
* second order optimization:

مسألة 1 (1st order GD) ، إنه بـ Small step

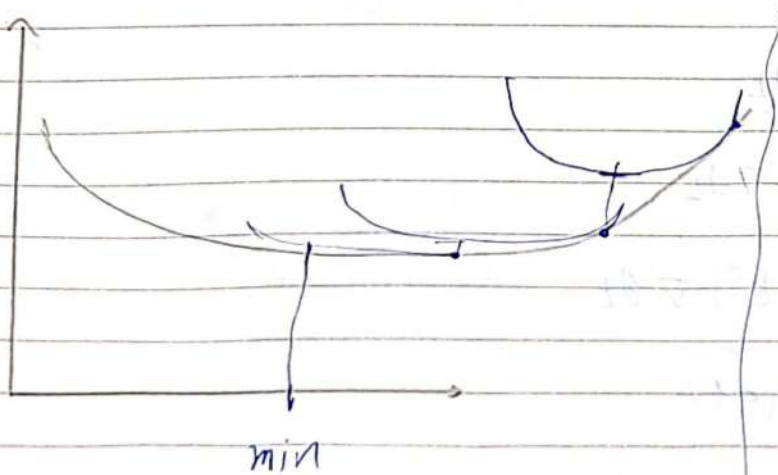
→ Approx. as (line)



لذلك، بدلاً من ذلك
Curve Approximation
(2nd order)



وخط واحد
one step
→ min



احتاج
3 step
لكننا نحتاج 3 خطوات
min



المميزات

- ① يأخذ (step) أكبر
- ② يوصل لـ min أسرع
- ③ يمكن محتاجش (learning rate)

الفكر

لـ باستخدام الـ (Hessian matrix)
يعرف درجة انحناء
سطح الـ (loss function)



assumptions :

Newton

$$f(x_{n+1}) = f(x_n) + f'(x_n)(x_{n+1} - x_n) + \frac{f''(x_n)}{2}(x_{n+1} - x_n)^2$$

when:

$$f'(x_{n+1}) = 0$$

$$0 = f'(x_n) + f''(x_n)(x_{n+1} - x_n)$$

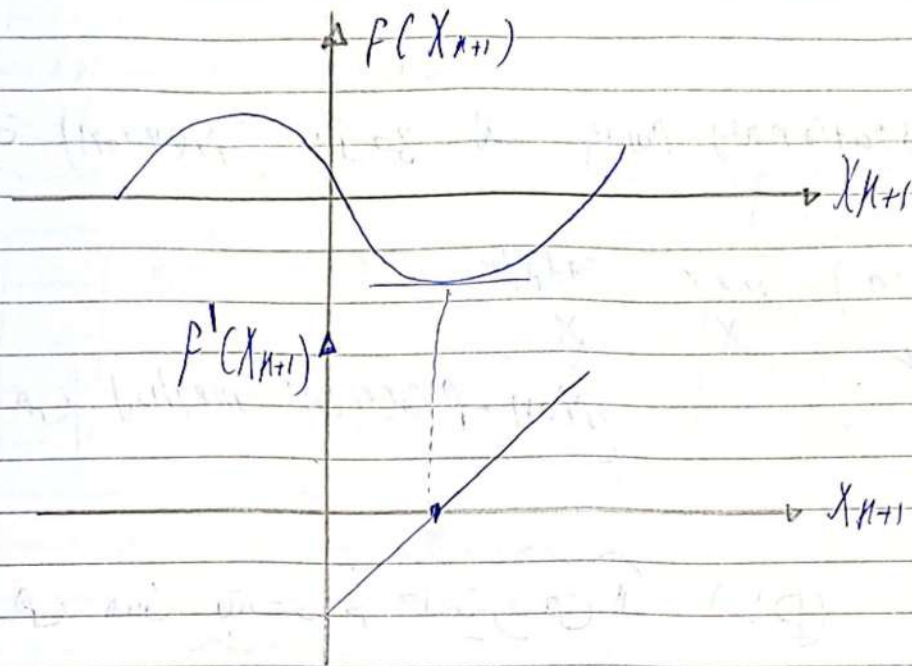
$$f'(x_n) + f''(x_n)(x_{n+1} - x_n) = 0$$

$$x_{n+1} - x_n = - \frac{f'(x_n)}{f''(x_n)}$$

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

↳ connection to Newton-Raphson method





$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)} \rightarrow \begin{matrix} g(x) \\ g'(x) \end{matrix}$$

حالات جيب الـ $\min$ و $\max$ و $f''(x_n)$ و $g'(x)$

لازم زوفا و $\sqrt{\text{root}}$ بتاع $f'(x_{n+1})$ و $g(x)$



(مسألة)

طريقة (Newton) بترجع إلى Stationary point

min max saddle
✓ x x

لذلك هي Non-practical method

لذلك هي ليست جيدة كثير في (DL)

ولكن، تستخدم في (Convex function) خفا

أو low order curves عموماً



$$X_{H+1} = X_H - H^{-1}(X_H) \nabla f(X_H)$$

-1

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}_{H+1} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}_H - \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 x_2} & \dots \\ \vdots & \ddots & \vdots \end{bmatrix}_H \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \end{bmatrix}_H$$

$H^{-1} \Rightarrow$ is computationally expensive

$$\underbrace{\underbrace{\mathcal{O}(n \cdot p^2)}_{\substack{\text{No. of} \\ \text{Samples}}} \rightarrow}_{\substack{\text{No. of} \\ \text{Parameters}}} H$$

$$\mathcal{O}(n \cdot p^3) \rightarrow H^{-1}$$



* Quasi-Newton Methods :

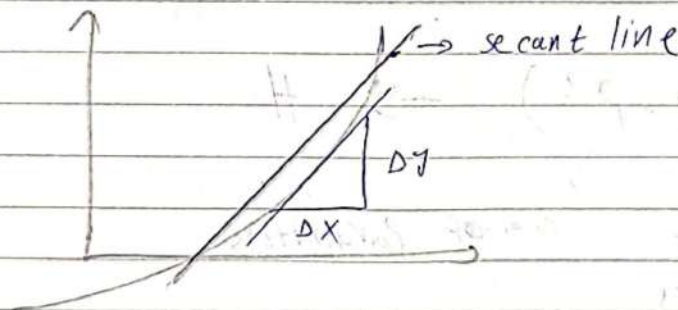
← بحاول اوجد Approximation لـ (H)

$$B_{K+1} \approx H_{K+1}$$

$$B_{K+1}^{-1} \approx H_{K+1}$$

* Intuition of Approximation

→ using secant method



$$f'(x_k) = \frac{\Delta f(x)}{\Delta x} = \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$



for 1st order :

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

$$= x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

↳ Approximation
of 1st derivative

for 2nd order :

$$\theta_{t+1} = \theta_t - \frac{J'(\theta_t)}{J''(\theta_t)}$$

$$J''(\theta_t) = \frac{J'(\theta_t) - J'(\theta_{t-1})}{\theta_t - \theta_{t-1}}$$

$$J''(\theta_t) (\theta_t - \theta_{t-1}) = J'(\theta_t) - J'(\theta_{t-1})$$

↳ Approximated



$$B(\theta_t)(\theta_t - \theta_{t-1}) = \nabla \theta_t - \nabla \theta_{t-1}$$

Condition :

$$\hookrightarrow \begin{matrix} B_t & \Delta & X_t & = & y_t \\ \downarrow & & \downarrow & & \downarrow \\ n & & n & & n \end{matrix}$$

Note :

For the 1st iteration:

assume:

$$\theta_{-1} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad \theta_0 = \begin{bmatrix} 0.001 \\ 0.001 \\ 1 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = I$$

1st iteration ممكن (استخدم في) (H) في



* Common algorithms:

- ① DFP (Davidon - Fletcher - Powell)
- ② BFGS (Broyden - Fletcher - Goldfarb - Shanno)
- ③ L-BFGS (limited-memory BFGS)



* BFGS :-

$$* S_k = \theta_{k+1} - \theta_k$$

$$* y_k = \nabla \theta_{k+1} - \nabla \theta_k$$

$$H_{k+1} = \left(I - \frac{S_k y_k^T}{y_k^T S_k} \right) H_k \left(I - \frac{y_k S_k^T}{y_k^T S_k} \right) + \frac{S_k S_k^T}{y_k^T S_k}$$

